



CS 431/631 Introduction to Big Data

Build and Train Deep Learning Models

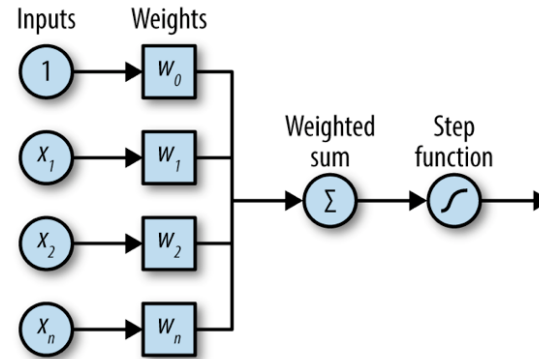
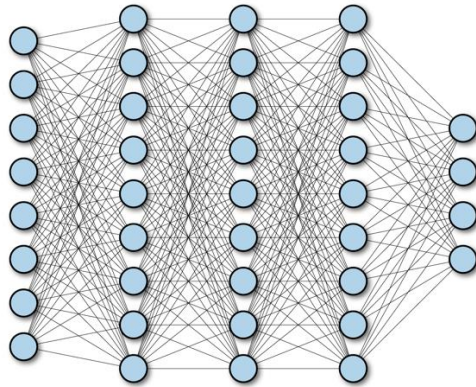
Some slides are borrowed from Yiran Chen, Hai Li, et. al.

Lei Yang
Computer Science & Engineering
University of Nevada, Reno



Before CNN

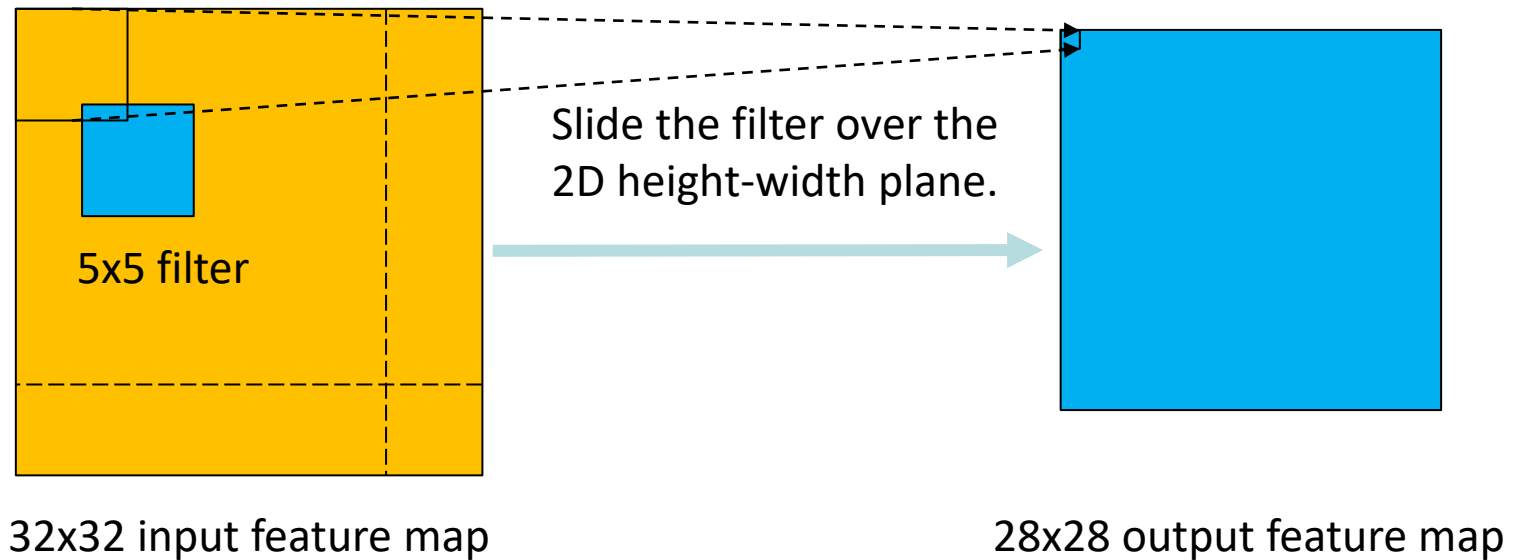
- Fully connected networks



- Fully connected networks on image recognition is difficult for the following reasons:
 - Images have very high-dimensional input features (e.g. $224 \times 224 \times 3$)
 - This can lead to the explosion of parameters as well as optimization difficulty

Convolution

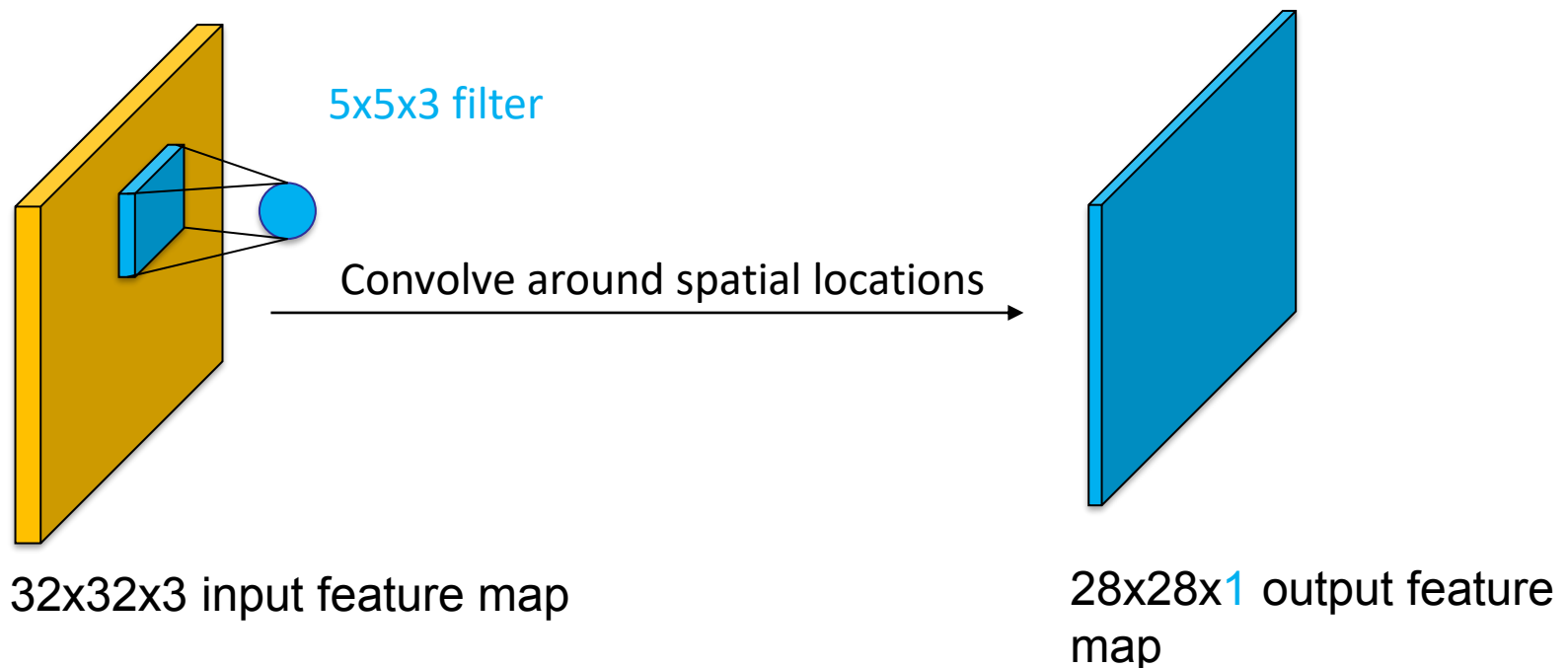
- Suppose the input image has only 1 channel. We add a **blue** filter on a 32×32 input feature map.



Convolution kernel with kernel size $K=5$

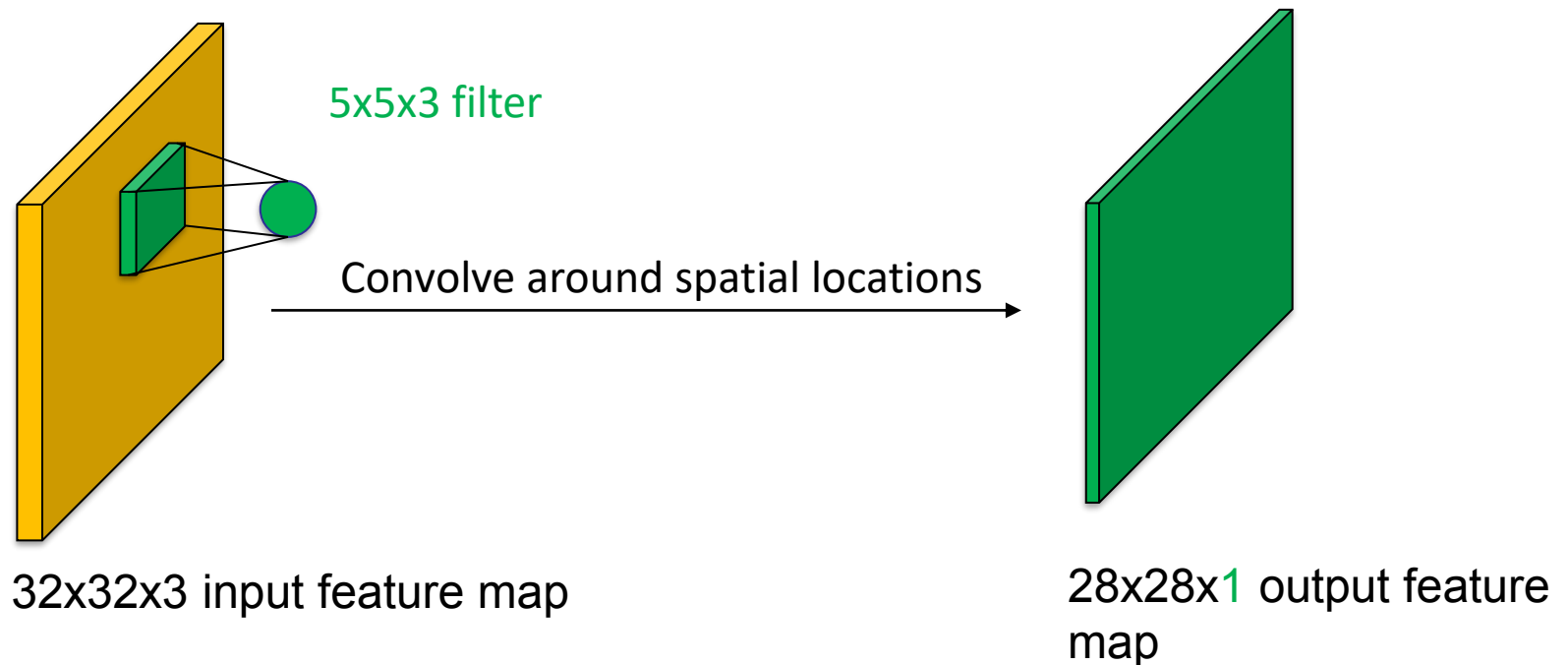
Convolution

- For an image with 3 channels (R,G,B), we expand the blue filter to a 3-D filter to extend the full depth of the input feature map.
- Slide over the image spatially and compute dot products to get the output feature map.
- Then extend the output feature map to 3-D for the additional channel dimension.



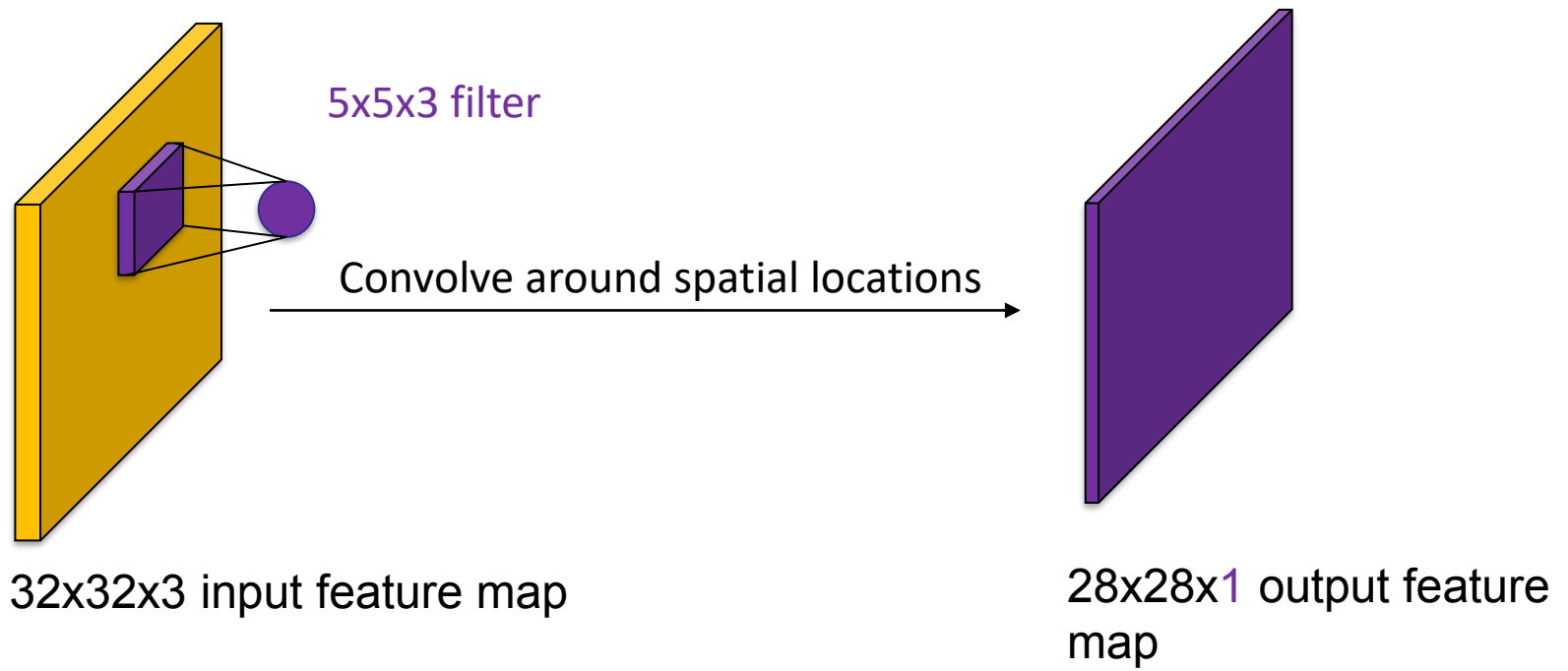
Convolution

- Let's add another 3-D **green** filter on the input feature map.
- Slide the **green** filter over the image spatially and obtain another 3-D output feature map.



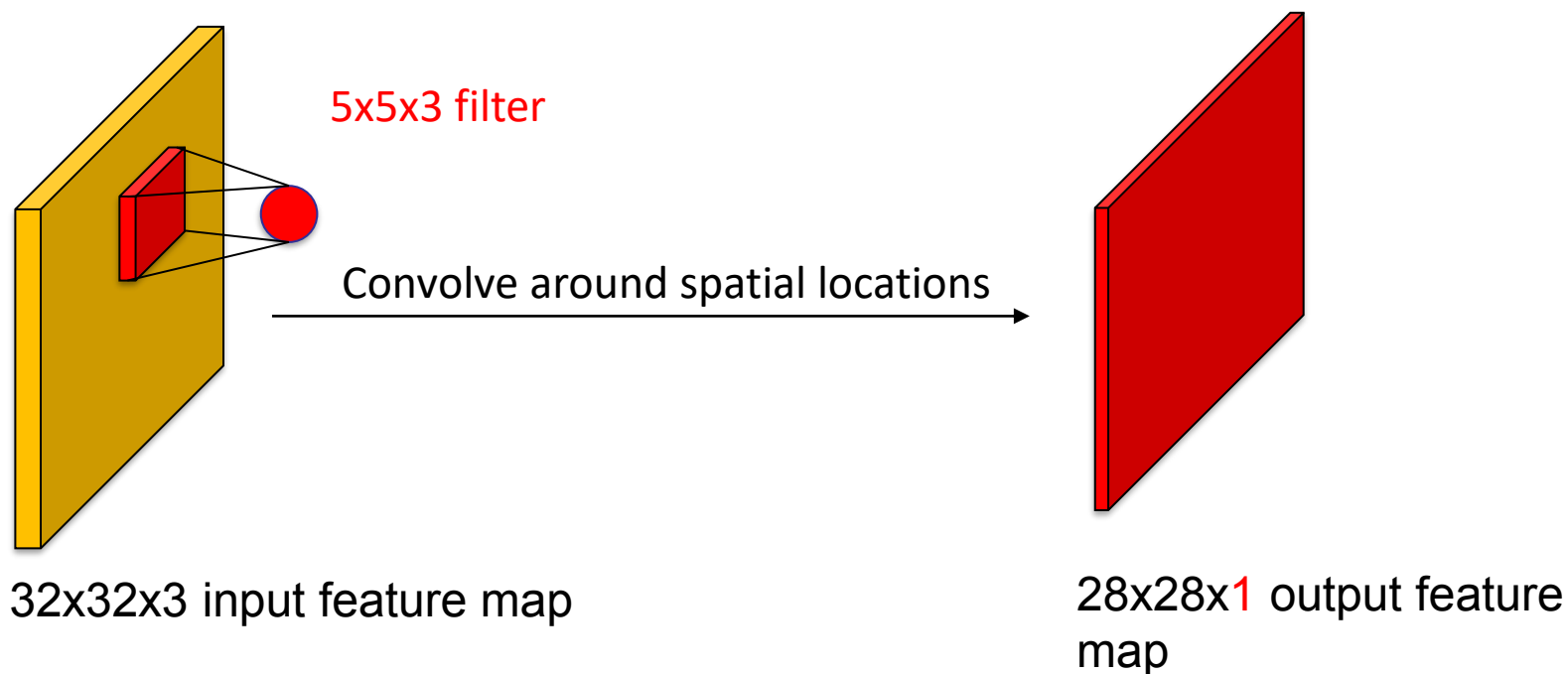
Convolution

- Let's add another 3-D **purple** filter on the input feature map.
- Slide the **purple** filter over the image spatially and obtain another 3-D output feature map.



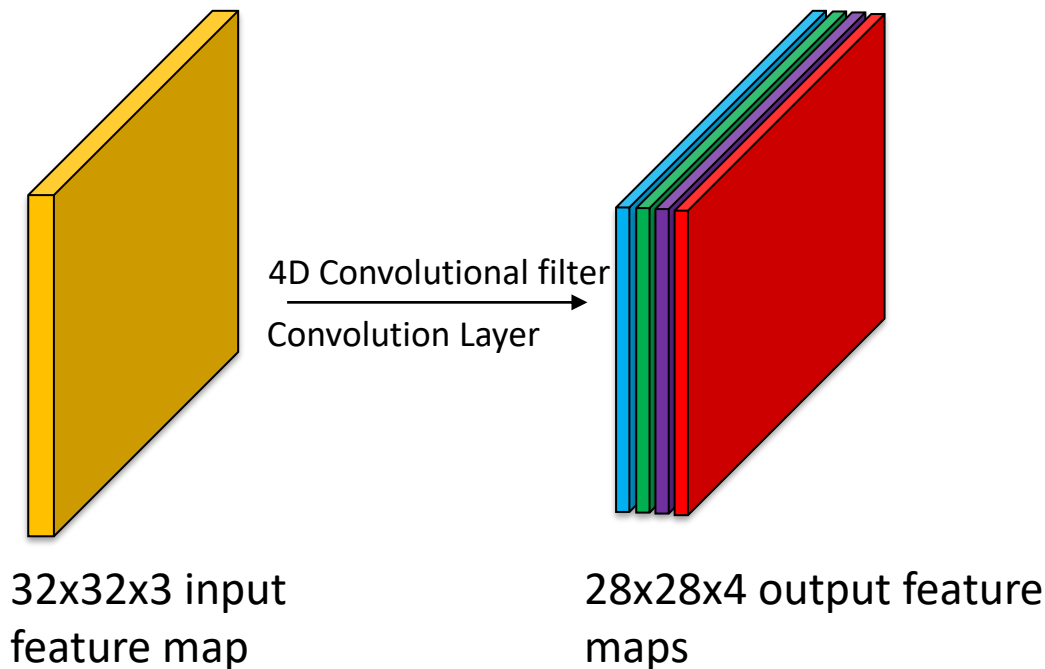
Convolution

- Let's add another 3-D **red** filter on the input feature map.
- Slide the **red** filter over the image spatially and obtain another 3-D output feature map.



Convolution

- Stack all the 4 3-D filters into a 4-D filter with shape $5 \times 5 \times 3 \times 4$.
- The output feature maps are also concatenated together into a 3-D output feature map with shape $28 \times 28 \times 4$.

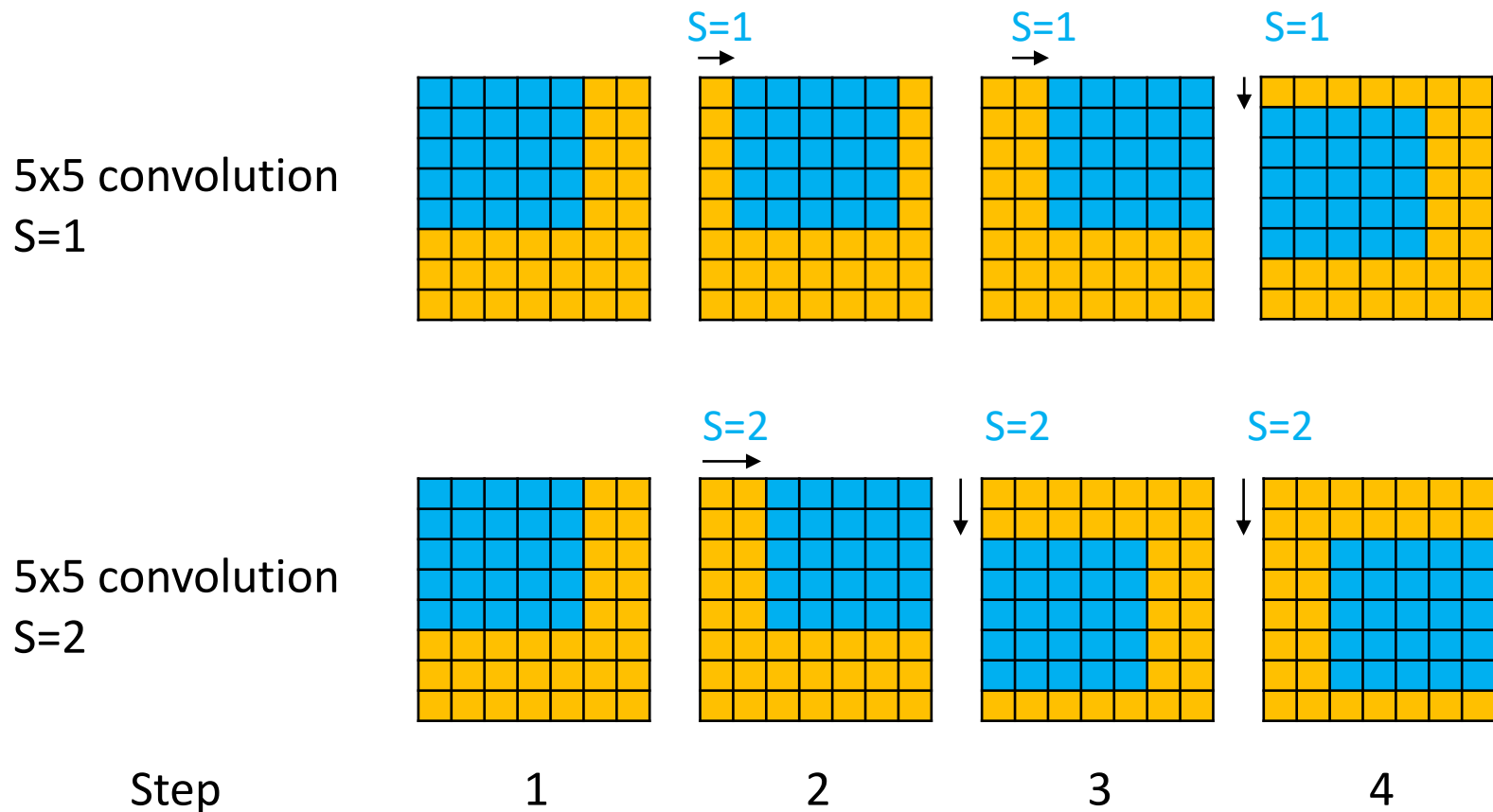


Convolutional Filter shape:
 $(H, W, I, O) \leftrightarrow (5, 5, 3, 4)$

H: height of convolutional filters
W: width of convolutional filters
I: number of input channels to the convolution operation
O: number of output channels after the convolution operation

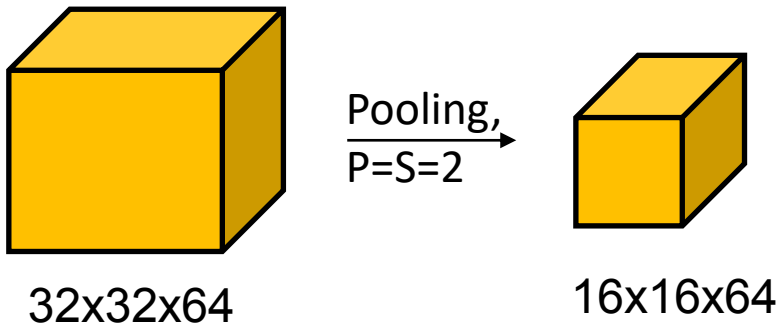
Convolution layer: strides

- Strides control the amount by which the filter slides over the input feature map. Use S to denote the strides of one convolution operation.



Pooling

- Pooling down-samples input features to extract a concise and manageable knowledge representation.



Note:

Pooling is conducted
independent of channels

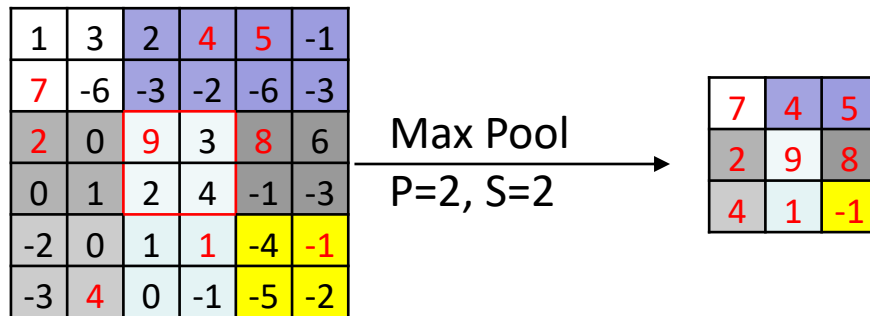
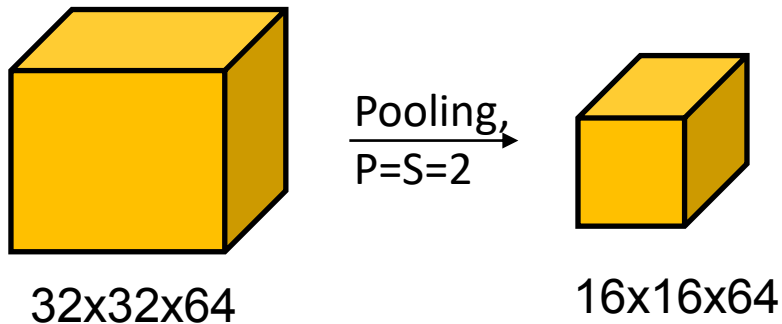
Pooling size (P)
the region to down-sample features according to pooling policy.

Strides (S)
The amount by which pooling region slides over the input feature map.

Generally, pooling size is equal to strides in the pooling layer.

Max pooling

- Max Pooling: select the maximum value in each region.



Pooling conducted along heights and width dimension of the feature map, not changing the depth.

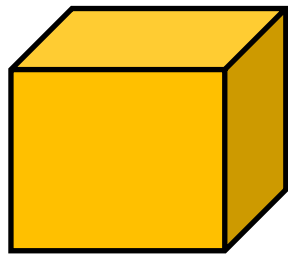
Pooling size (P)
the region to down-sample features according to pooling policy.

Strides (S)
The amount by which pooling region slides over the input feature map.

Generally, pooling size is equal to strides in the pooling layer.

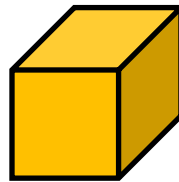
Average pooling

- Average pooling: average the response in each region.



32x32x64

Pooling,
P=S=2



16x16x64

Similar process as Max pooling, changing the operation from maximize to average.

1	3	2	4	5	-1
7	-6	-3	-2	-6	-3
2	0	9	3	8	6
0	1	2	4	-1	-3
-2	0	1	1	-4	-1
-3	4	0	-1	-5	-2

Average Pool
P=2, S=2

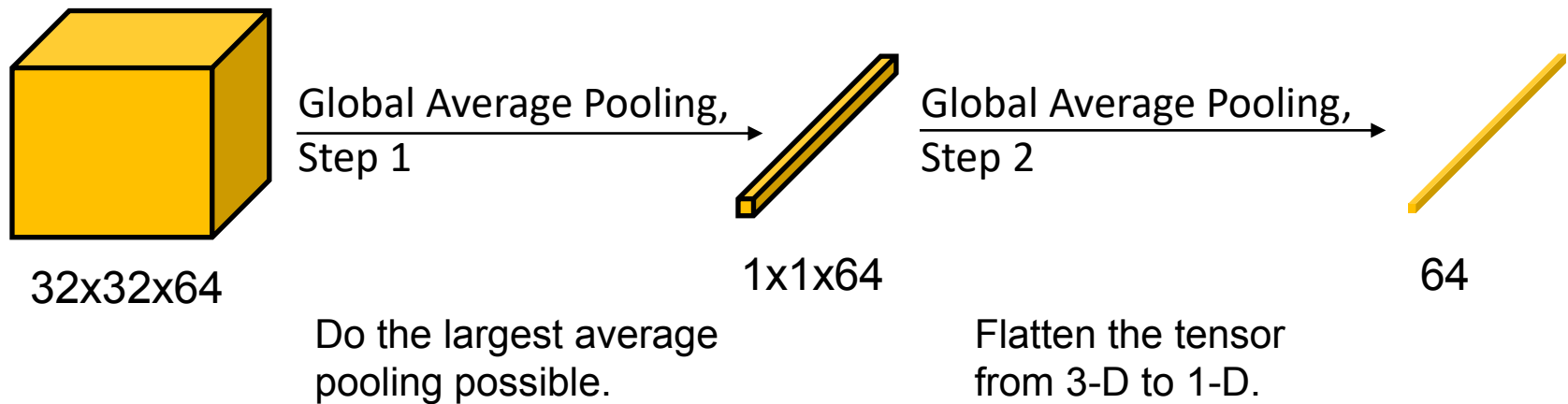
1.25	0.25	-1.25
0.75	4.50	2.50
-0.25	0.25	-3.00

Average pooling consumes extra computation.

$$((-2)+(-3)+0+4)/4=-0.25$$

Global average pooling

- Pool all of the spatial dimensions and get a flattened tensor.

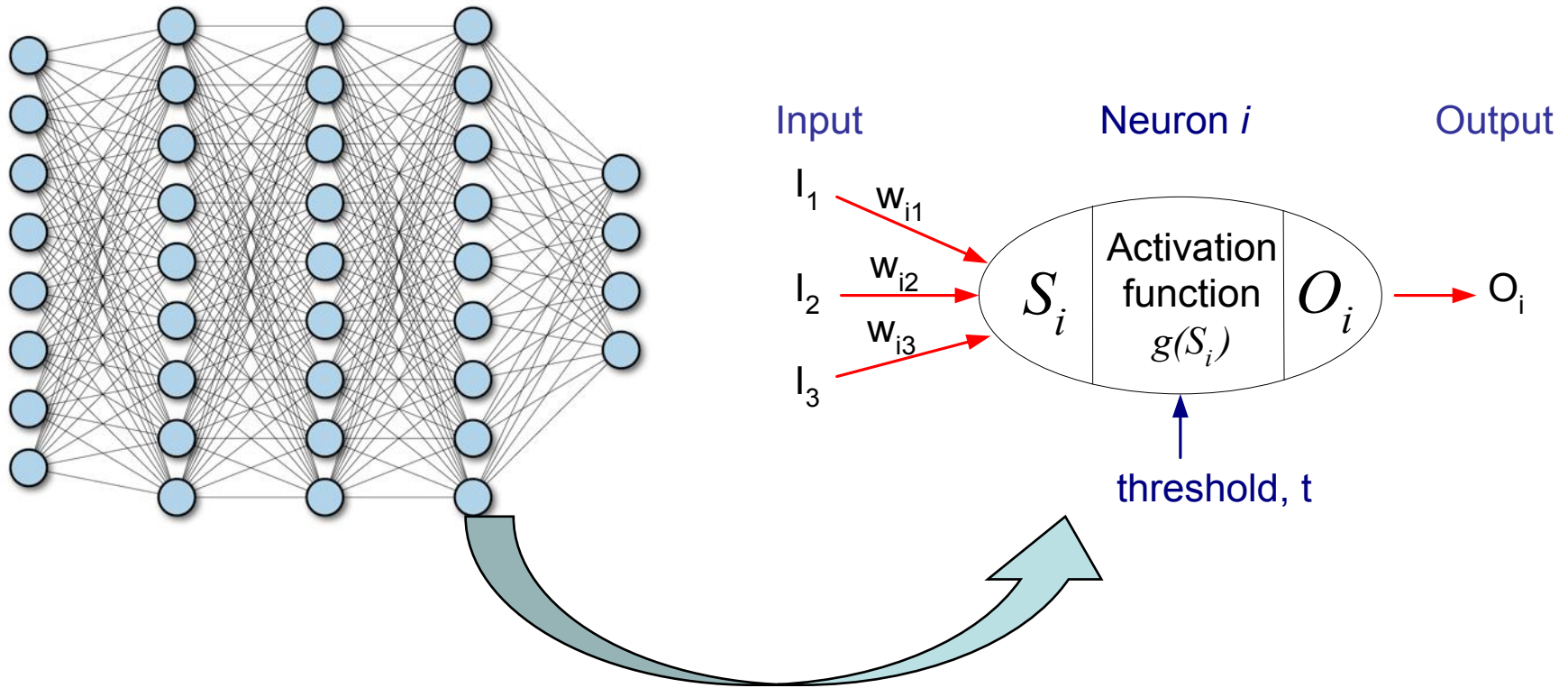


1	3	2	4	5	-1
7	-6	-3	-2	-6	-3
2	0	9	3	8	6
0	1	2	4	-1	-3
-2	0	1	1	-4	-1
-3	4	0	-1	-5	-2

Global Average Pool

0.56

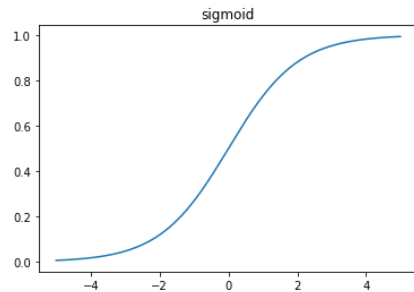
Fully connected networks



Activation function

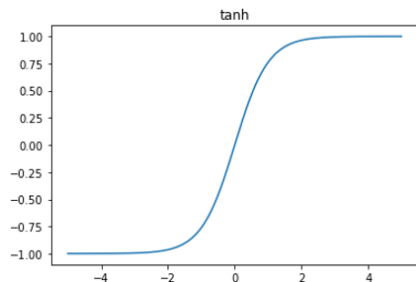
- Each convolution layer is followed by an activation function to create **non-linear** complex functional mappings between inputs and predictions.
- Activation gives DNNs the ability to **learn complicated representations**.
- Without any non-linear activation, a multi-layer neural degrades to a single-layer network as multi-layer linear transformation can be applied within one linear layer.
- Output activations are called **feature/activation maps or output channels**.

Overview of activation functions



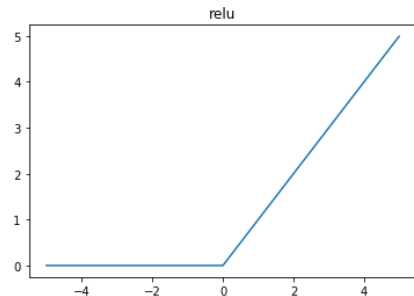
Sigmoid

$$a(x) = \frac{1}{1 + e^{-x}}$$

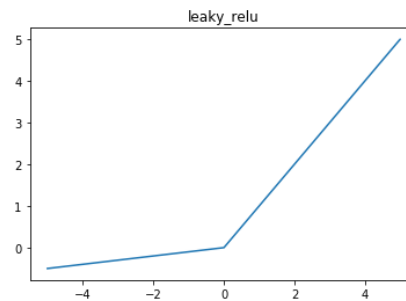


Tanh

$$a(x) = \tanh(x)$$

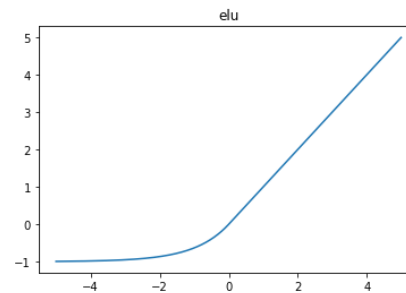


Rectified linear unit(ReLU)
 $a(x) = \max(0, x)$



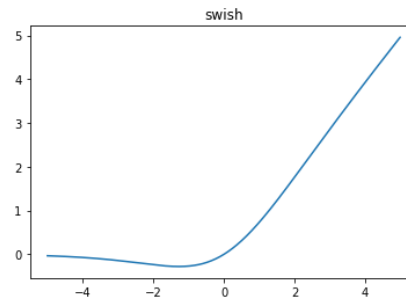
Leaky ReLU

$$a(x) = \max(\alpha x, x)$$
$$\alpha \in (0,1)$$



ELU

$$a(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$
$$\alpha \in (0,1)$$



Swish

$$a(x) = x\sigma(x)$$

Sigmoid

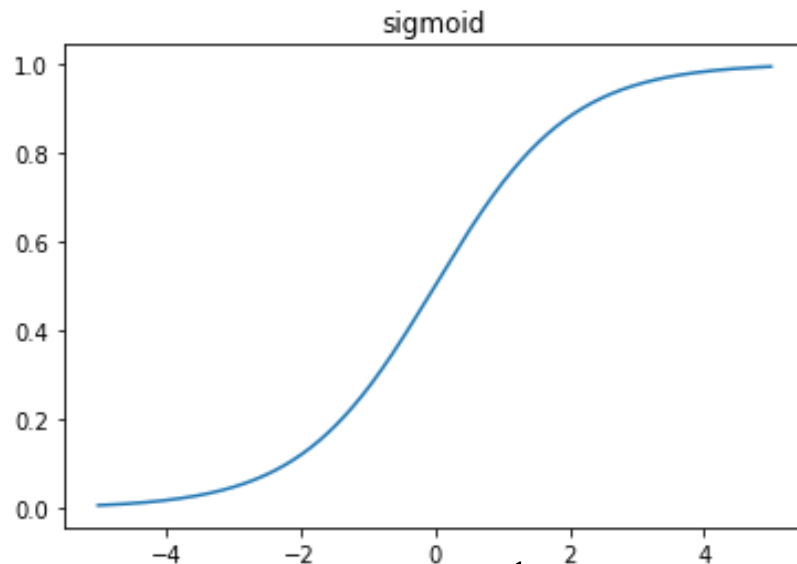
- Gradients are close to zero when the output neuron gets saturated.
- Usually, sigmoid activation is only used at the **end of the last layer** to rescale and bound the prediction logits.

Advantage:

- Bounded output

Problems:

- Gradient vanishing
- Non zero-centered output



$$a(x) = \frac{1}{1 + e^{-x}}$$

Tanh (hyperbolic tangent function)

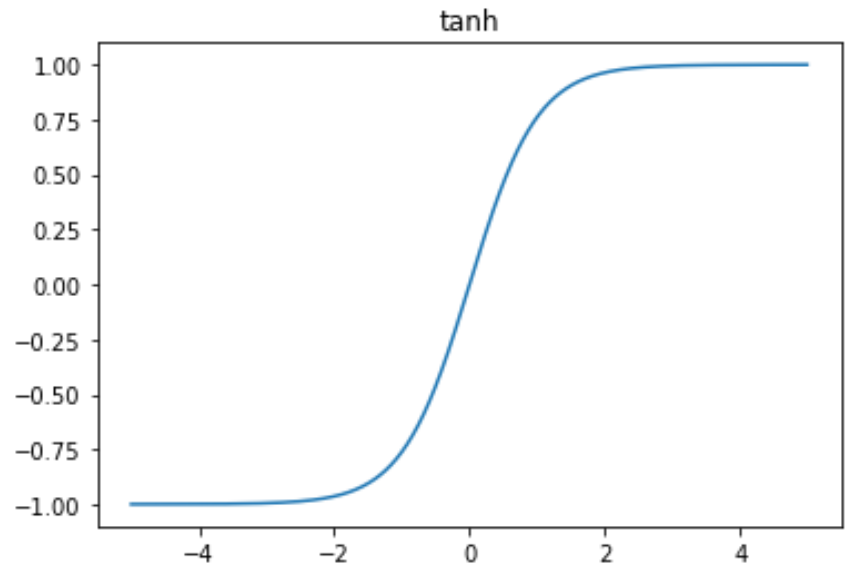
- Gradients are close to zero when the output neuron gets saturated.

Advantage:

- Bounded & zero-centered output

Problem:

- Gradient vanishing



Rectified linear unit (ReLU)

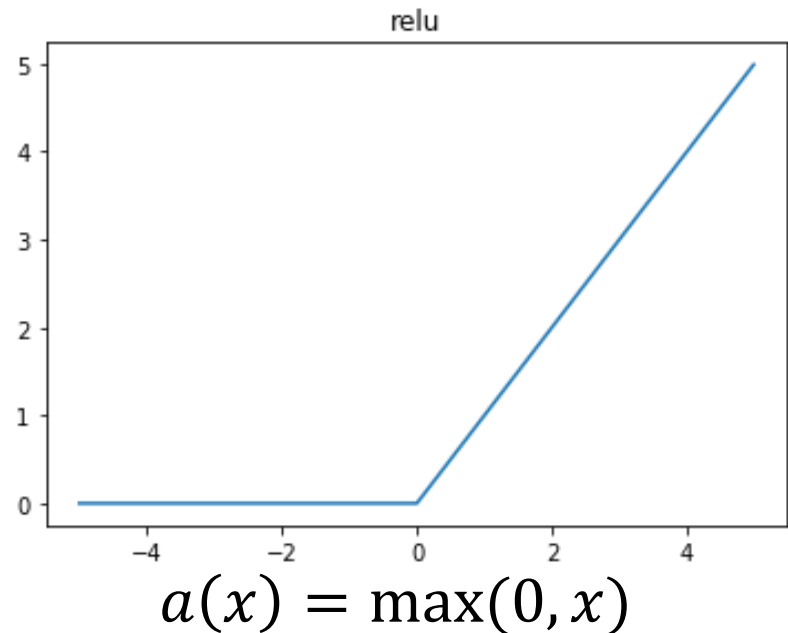
- The gradient is constant as long as the neuron is activated.
- ReLU activation can be used for **every layer except for the last one**.

Advantages:

- + Computationally efficient
- + Biologically plausible

Problems:

- Dead Neurons
- Non zero-centered output.



Leaky ReLU

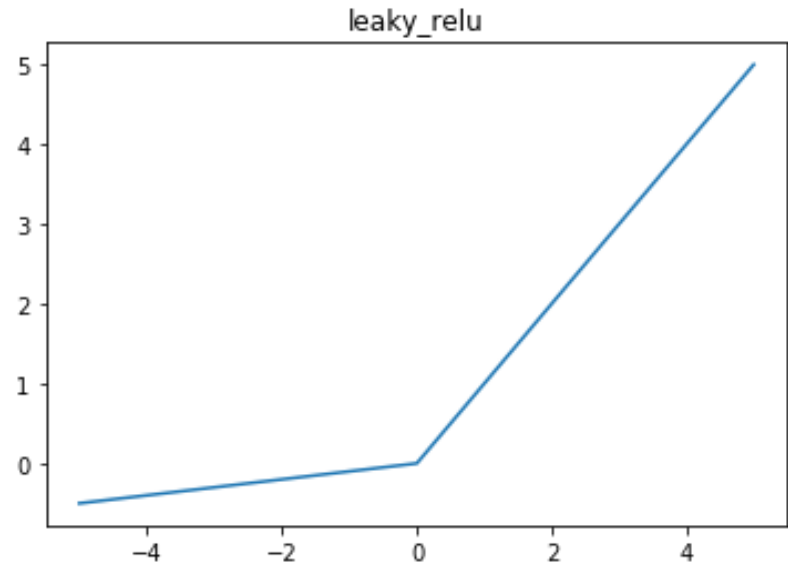
- Stable gradients are provided when neuron is in **dead area** (<0).

Advantages:

- + Computationally efficient
- + No dead neurons

Problems:

- Non zero-centered output.
- May take more time to reach convergence.



$$a(x) = \max(\alpha x, x)$$
$$\alpha \in (0,1)$$

SoftMax

Sigmoid activation is used in binary classification. For **multi-class** classification, we use the SoftMax activation. It outputs a **bounded** logit for prediction.

$$y = \begin{bmatrix} 2.0 \\ 1.0 \\ 0.1 \end{bmatrix} \longrightarrow S(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}} \longrightarrow y = \begin{bmatrix} 0.66 \\ 0.24 \\ 0.10 \end{bmatrix}$$

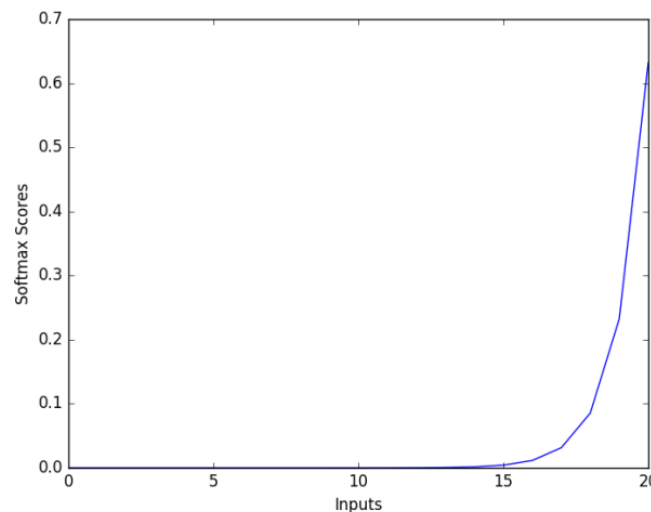
Similar to sigmoid activation, SoftMax activation is **only** used at the end of the last layer.

Advantage:

- Bounded output

Problems:

- Gradient Vanishing
- Non zero-centered output



Exponential linear unit (ELU)

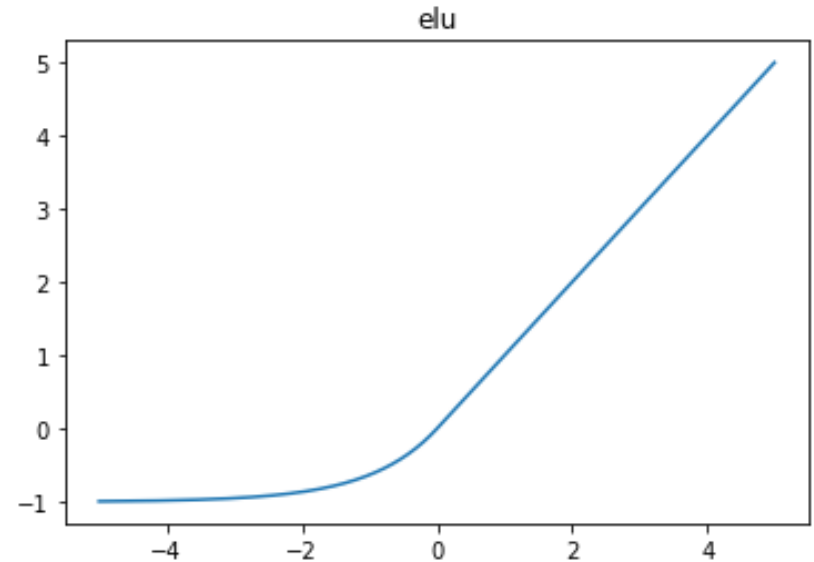
ELU uses the exponential operation to increase the **representation power** of DNNs. ELU provides more robustness to noise in the negative saturation part.

Advantages:

- + Closer to zero-mean outputs
- + Neural networks insensitive to weight initialization
- + Faster convergence

Problem:

- Computationally inefficient.



$$a(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

$\alpha \in (0,1)$, usually we use $\alpha=1$

Swish

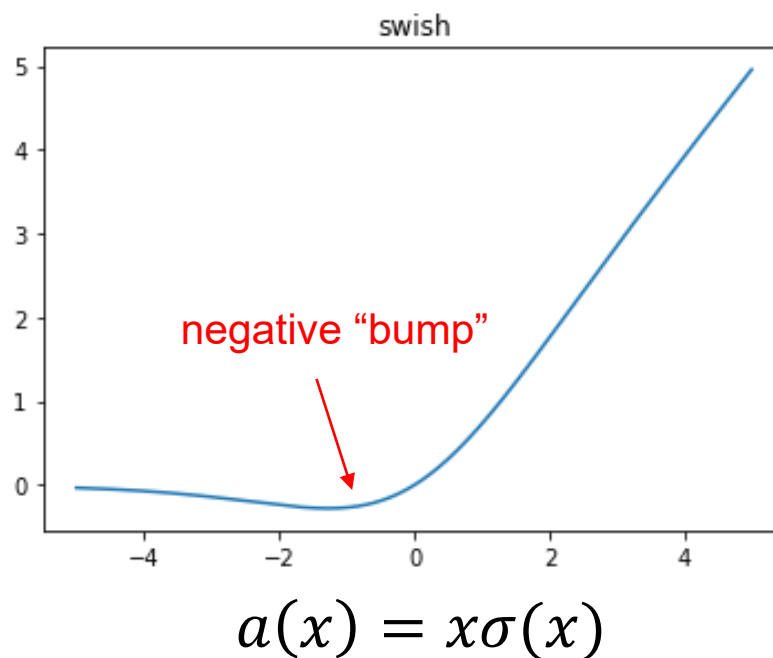
- Swish is an activation function found by activation search. The non-monotonic “**bump**” of Swish boosts the performance of this activation function.

Advantages:

- + Increase representation power for DNNs, especially small models.
- + Lead to performance gain by added representational power from the negative “bump”.

Problem:

- Computationally inefficient.



Neural network design

- What should be a good network design?
“Thinner at the top, wider at the bottom”



Good neural network design. Wider architectures at the bottom compensate for the information loss induced by smaller feature map.

Not recommended. It is unclear that how information is accumulated and preserved within filters.

Bad design. There is no information left before the final output layer.

wide/thin: number of output filters
tall/short: feature map size

Neural network design

How to choose the number of filters in a convolutional neural network?

- By convention, the filter numbers C should be in the form of 2^n .
- Number of filters should grow (most likely by a factor of 2) after each down-sampling modules (e.g. MaxPool).

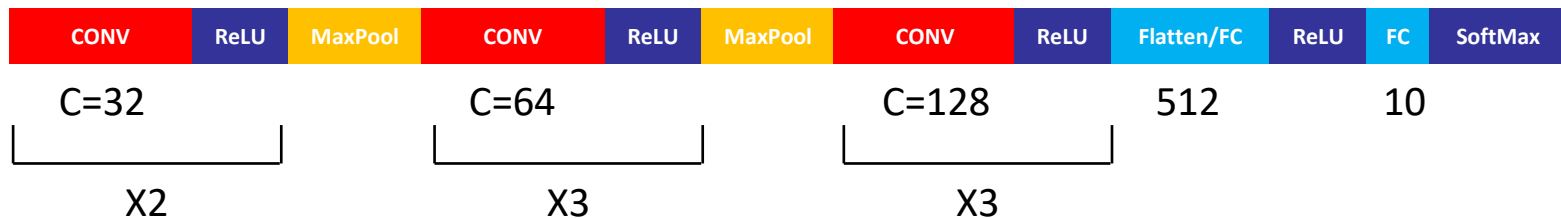


- For example, the above configuration is good enough for training on MNIST.

Neural network design

How many convolutions should we insert between each pair of the down-sampling modules?

- More convolutions should be inserted between later pairs of down-sampling modules.
- Do not insert too many operations to form very deep convolutional networks.



Neural network design

Which activation function should we use?

- It is always good to start with ReLU. You can switch to other kinds of activations and expect higher performance gain when ReLU works for your DNN design.
- Using activation functions other than ReLU increases the difficulty of debugging as they can try to 'converge' any type of neural network.
 - For example, even if your neural network is not configured correctly, ELU tends to converge but only reaches a bad performance.
- Use SoftMax activation function in the final output layer.

Loss function

Cross Entropy (Softmax) loss (classification)

$$s = f(x_i; W)$$

$$L_i = -\log P(Y = y_i | X = x_i) = -\log \frac{e^{s_{y_i}}}{\sum_j e^{s_j}}$$

Suppose we have N classes. What is the value of the loss function if the weights $\mathbf{W}$ are initialized with evenly random numbers?

log N . Because random guess is performed when DNN has learned nothing. All s will be approximately equal.

It is recommended to check the initial loss value before you start training. For example, for CIFAR-10 dataset with $N=10$ classes, the initial loss should be around 2.3.

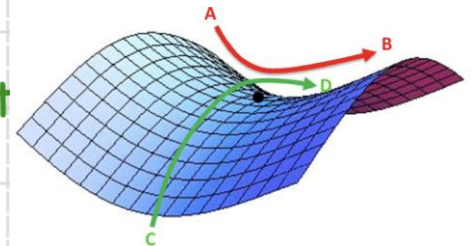
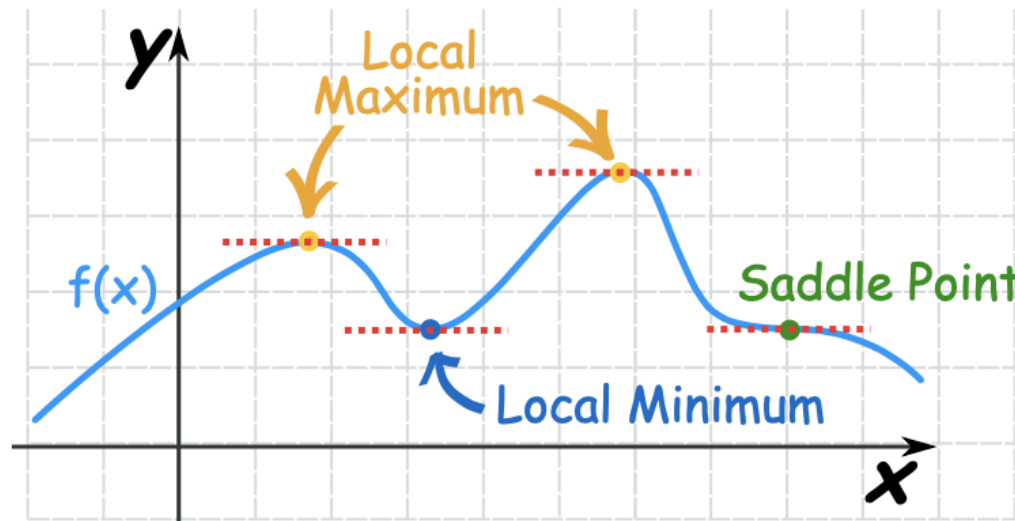
Stochastic gradient descent (SGD)

Weights are updated by the computed vanilla gradient.

$$W^{t+1} = W^t - \alpha \nabla_W L(W^t)$$

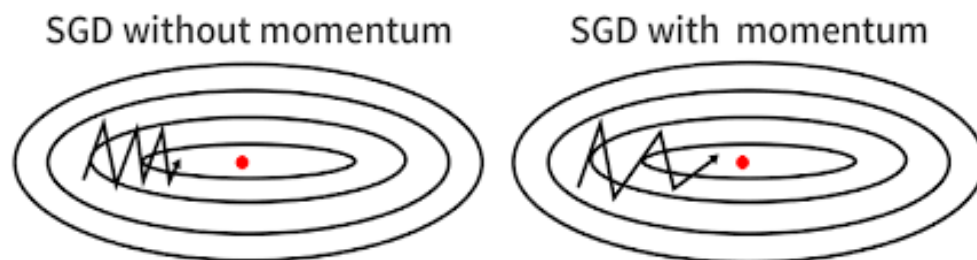
α : Learning rate

SGD is likely to get stuck into local minimum and saddle point.



SGD with momentum

‘Momentum’ is computed and accumulated to speed up the converging process.



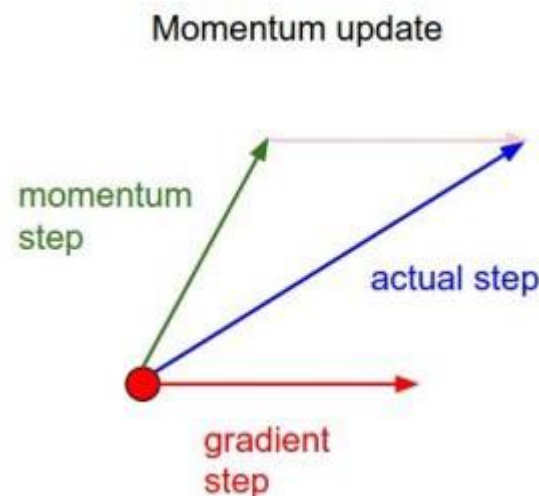
SGD with momentum avoids saddle point by ‘overshooting’ the saddle point with accumulated momentum.

Advantages:

- + Little computational cost during weight update
- + Usually used to reach the state-of-the-art performance

Problems:

- Requires more tuning time
- May overshoot the global minimum.



SGD with Nesterov momentum

Accumulate the momentum and 'look ahead': Mix the **current weight parameter** with **velocity** to compute the gradient.

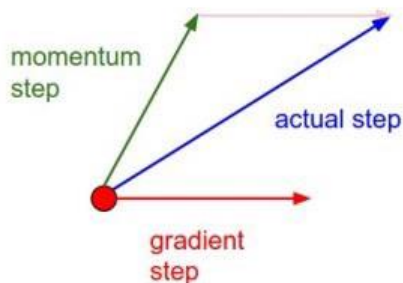
$$v^{t+1} = \mu v_t - \alpha \nabla_W L(W^t + \mu v_t)$$
$$W^{t+1} = W^t + v^{t+1}$$

α : Learning rate
 μ : Momentum (factor)

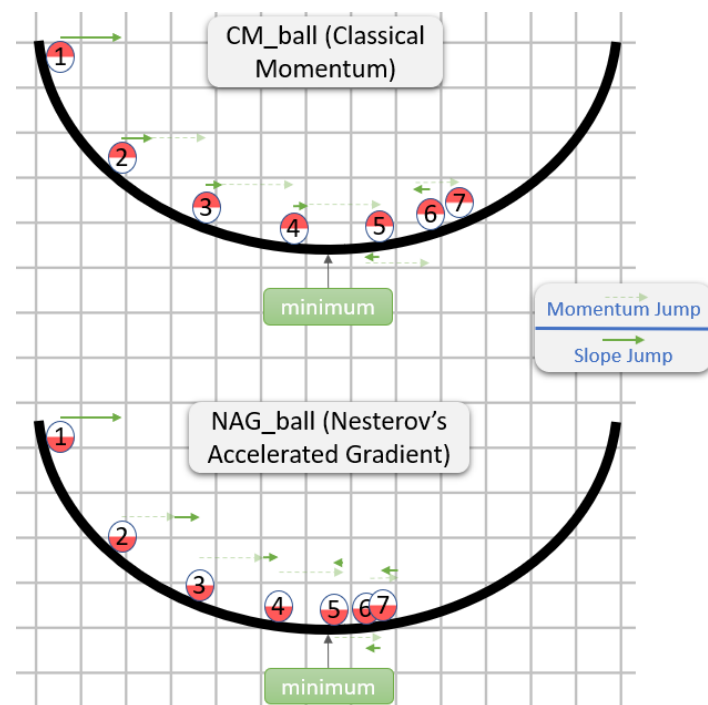
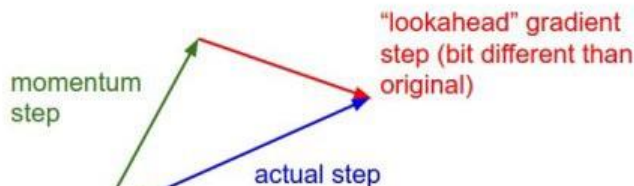
Advantages:

- + Avoid oscillations during optimization
- + Reach state-of-the-art performance

Momentum update

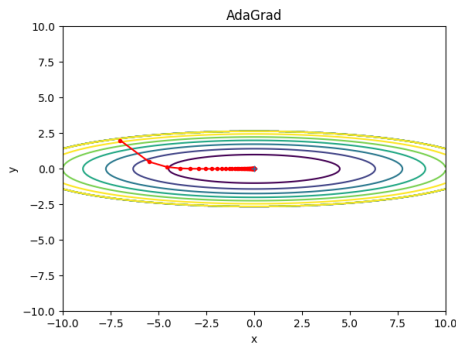


Nesterov momentum update



Adagrad (Adaptive gradient)

To **speed up** the training process, adaptive learning rate can be used for **each weight parameter**.



$$W^{t+1} = W^t - \frac{\alpha \nabla_W L(W^t)}{\sqrt{G} + \epsilon}$$

α : Learning rate
 ϵ : Small number to avoid zero division

$$G = \sum_{i=0}^t \left(\nabla_W L(W^i) \right)^2$$

Advantage:

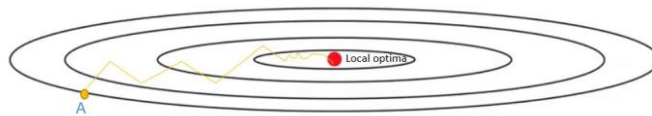
+ Increase the smoothness of SGD update

Problem:

- Accumulation of gradients in the denominator shrinks learning rate too quickly.

RMSprop (root mean square propagation)

To address the problem in Adagrad, RMSprop uses a **moving average** of incoming gradients as the denominator.


$$\begin{aligned} \rho^{t+1} &= \mu G^t + (1 - \mu) \left(\nabla_W L(W^t) \right)^2 \\ W^{t+1} &= W^t - \frac{\alpha \nabla_W L(W^t)}{\sqrt{\rho^{t+1} + \epsilon}} \end{aligned}$$

α : Learning rate
 μ : Momentum (factor)
 G^t : gradient norm at time step t .
 ϵ : Small number to avoid zero division

Advantages:

- + Prevent learning rate shrinkage
- + Good for recurrent neural networks

Problem:

- Oscillation at the end of convergence.

Adam (adaptive moment estimation)

Combine the features of **SGD with momentum** and **RMSprop Optimizer**.

- $g_t = \nabla_W L(W^t)$
- $m_{t+1} = \beta_1 m_t + (1 - \beta_1) g_t$ first momentum estimate
- $v_{t+1} = \beta_2 v_t + (1 - \beta_2) g_t^2$ second momentum estimate
- $\hat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}$ bias-corrected first-order momentum estimate
- $\hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}$ bias-corrected second-order momentum estimate
- $W^{t+1} = W^t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$ update, ϵ is a small number to avoid zero division

The use of first order and second order statistics gives Adam the **fastest convergence speed**. However, Adam is extremely unstable and oscillates loss values at the end of convergence. This may lead to some performance loss compared to SGD-based optimizers.

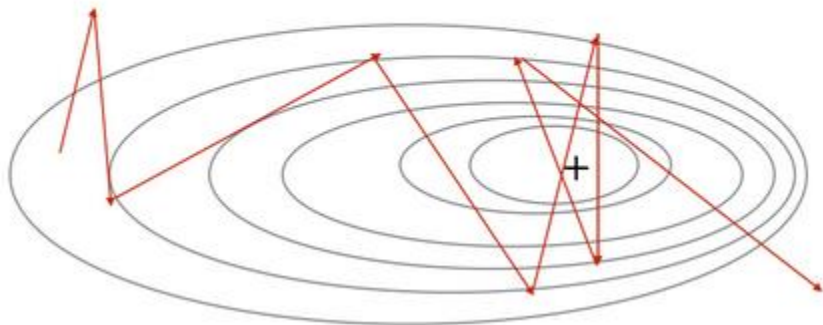
Which optimizer should we use for training?

- **Adam** is a good default choice in many cases because it has the fastest convergence speed.
- **SGD with momentum/Nesterov momentum** can outperform **Adam**, but it takes a longer time to tuning the weight parameters and reach convergence.

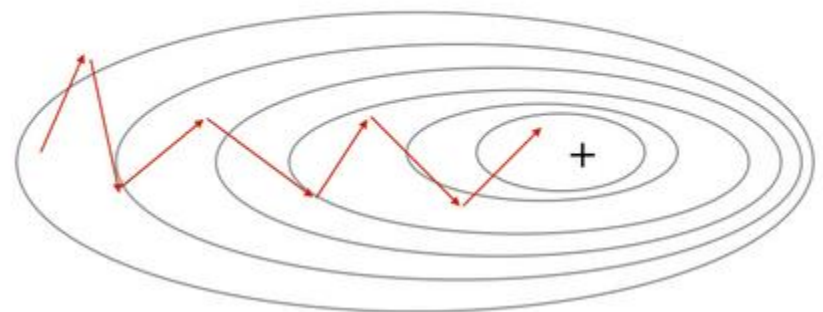
Gradient clipping

- To solve the problem of **exploding gradients**, we introduce gradient clipping to stabilize the training process.
- Gradient Clipping **removes the outliers** in the gradient and caps them within a given range. It is a common practice to clip the gradient values to $(-2, 2)$ to stabilize training for CNNs.

Without gradient clipping



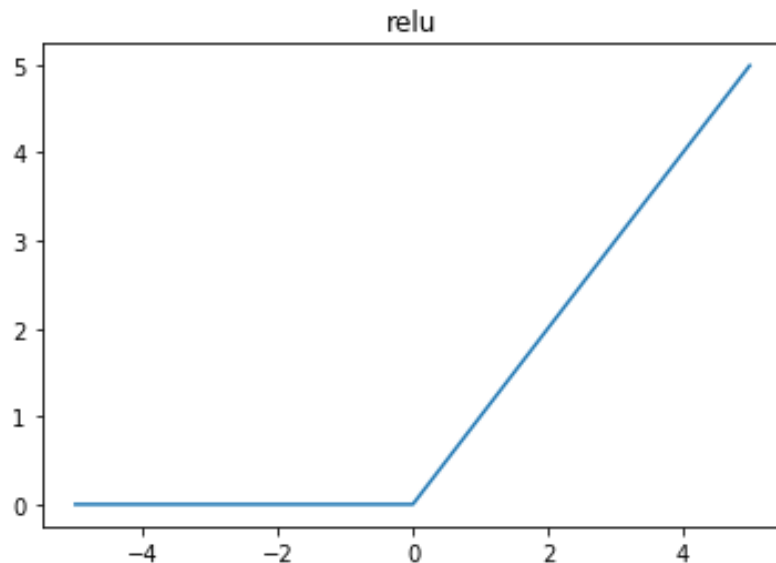
With gradient clipping



Weight initialization

- Without weight initializations, DNN can be tricky to train.

Suppose we use ReLU as activation function for all of the DNN layers. What will happen if all of the weight parameters and bias parameters are set to 0 at the beginning of training?



Answer:

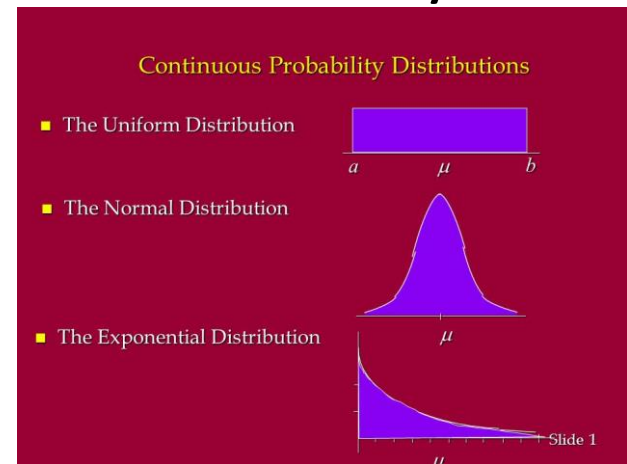
All outputs are stuck at 0 and all neurons are 'dead' forever.

The weights will never be updated despite of the incoming training data.

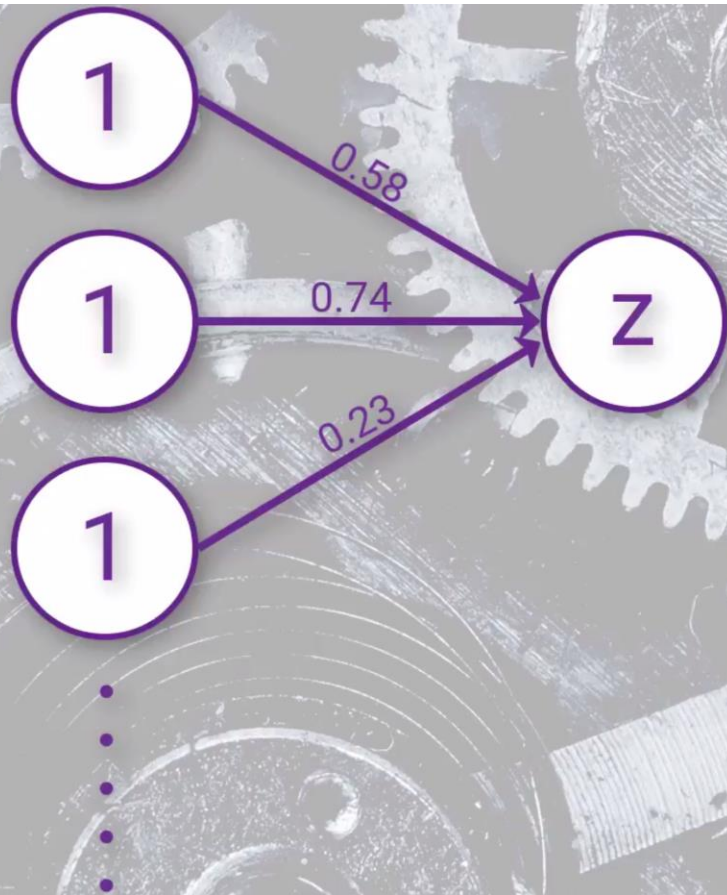
Weight initialization is needed to activate some of the neurons to generate some non-zero outputs and start the weight update process.

Random initialization

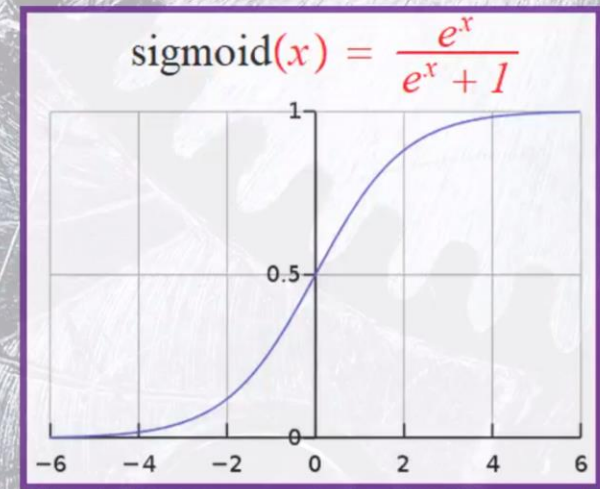
- Random initialization is performed to start the learning process. Usually, we randomly initialize the weight parameters to have **zero mean** and **small standard deviation** (e.g. 0.03). There are two commonly used random initializers:
 - Random uniform initializer
 - Random normal initializer
- Random initialization does NOT utilize the information of network structure. Therefore, unless carefully tuned, this method does not help improve the performance.



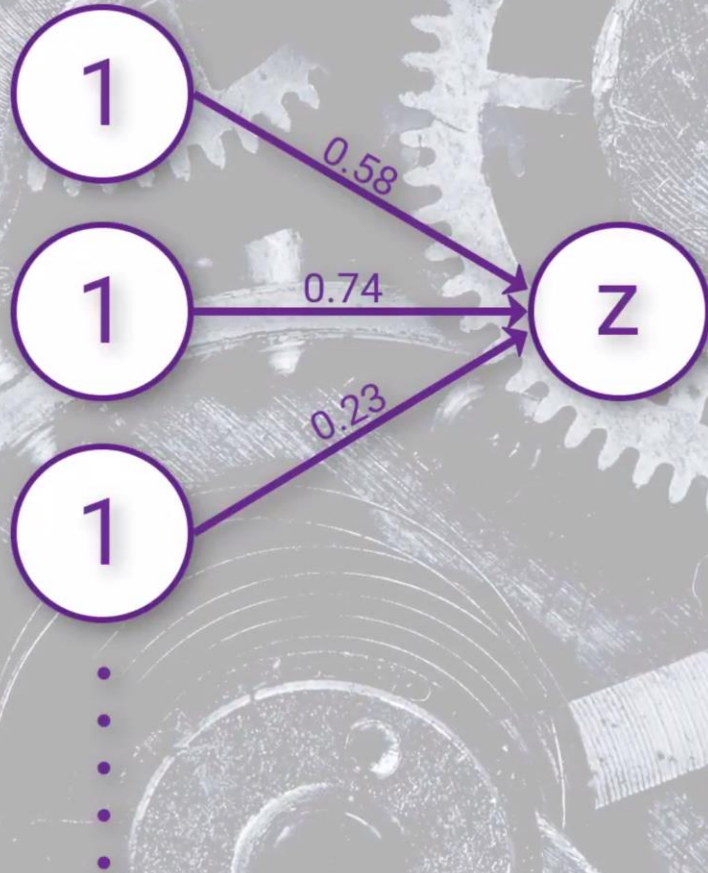
Random initialization



$$\begin{aligned}\text{var}(z) &= 250 \\ \text{stdv}(z) &= \sqrt{250} \\ &= 15.811\end{aligned}$$



Xavier initialization



Xavier Initialization

$$\text{var}(\text{weights}) = \frac{2}{n}$$

$$\text{weight} * \sqrt{\frac{2}{n}}$$

$$\text{var}(\text{weights}) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

Xavier initialization

To utilize the information of network structures, weights are initialized according to different **input units and output units**. There are two kinds of Xavier initializers.

- Xavier (Glorot) normal initializer

$$W \sim N\left(0, \sqrt{\frac{2}{fan_in + fan_out}}\right)$$

- Xavier (Glorot) uniform initializer

$$W \sim U\left[-\sqrt{\frac{6}{fan_in + fan_out}}, \sqrt{\frac{6}{fan_in + fan_out}}\right]$$

The assumption for Xavier Initializer holds for linear/quasi-linear activations. Therefore, Xavier initialization makes sense mathematically. However, this method does not work for newer model designs with concatenations.

Xavier initialization: uniform vs. normal

- Under most cases, distribution of initialized DNN weights is **preserved after training**. Therefore, it is important to decide whether we want to use uniform weight initializer or normal weight initializer.
- We recommend to use the **normal weight initializer** as it is more natural. In PyTorch, the default weight initialization method is *xavier_uniform*. Most neural networks obtain performance gain if changing the initialization method to *xavier_normal*.

Variance scaling (MSRA initialization)

Scale the variance of current weights by only the number of **input** units. This initialization favors DNN designs with **changeable output filters** (e.g. filter concatenations and additions)

For fully-connected layers with fan_in input units, MSRA initializes the weights by

$$W \sim N\left(0, \sqrt{\frac{1}{fan_in}}\right)$$

For convolutional layers with $K \times K$ kernel and C input channels, MSRA initializes the weights by

$$W \sim N\left(0, \sqrt{\frac{1}{K \times K \times C}}\right)$$

L-norm regularization

L-norm regularization address the **overfitting problem** by adding a term to **penalize for large weights**.

$$L(W) = CE(X, Y; W) + \lambda R(W)$$

Cross entropy loss

Regularization loss

- **L1 (Lasso) regularization**

$$R(W) = \sum_{i,j} |W_{ij}| = \|W\|_1$$

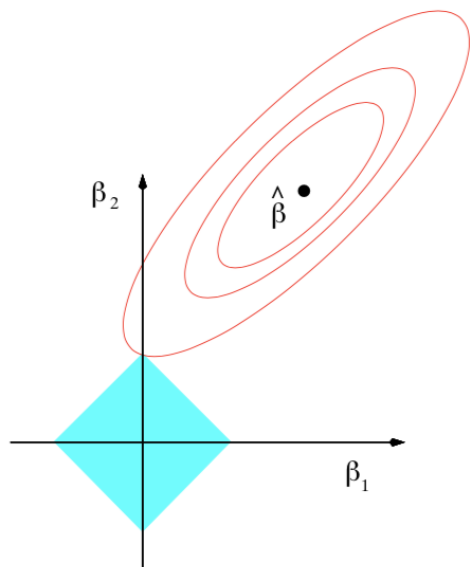
- **L2 (Ridge) regularization**

$$R(W) = \sum_{i,j} W_{ij}^2 = \|W\|_2^2$$

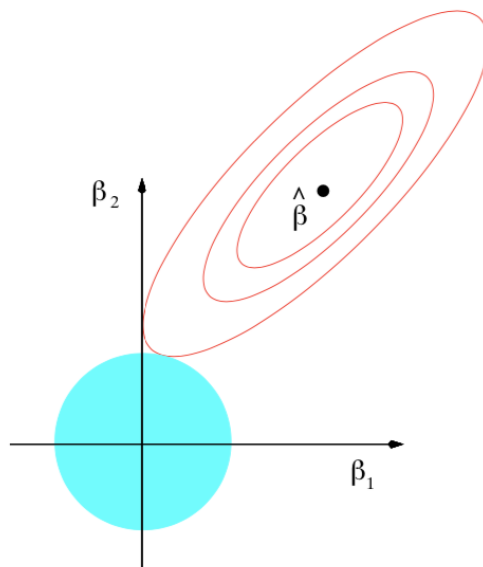
L-norm regularization

Visualization of L-norm regularization

Assume $W = \{\beta_1, \beta_2\}$.



L1 (Lasso)
regularization



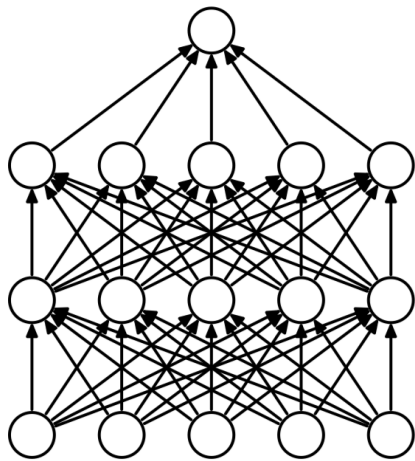
L2 (Ridge)
regularization

Since the loss surface of L1 regularization has spike at sparse points, applying L1 (Lasso) regularization during training is more likely to lead to **sparse weights**, while L2 (Ridge) regularization tends to produce **small weights**.

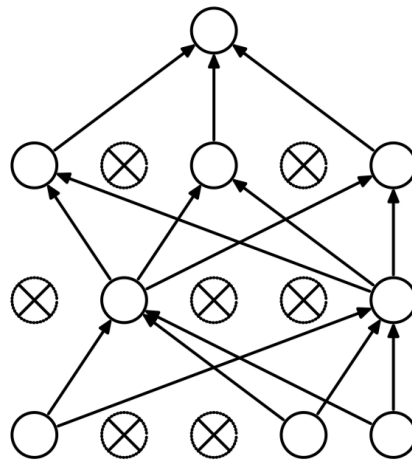
Evenly penalize all weights Penalize large weights more

Dropout

Randomly set some neurons to zero to drop units along with their connections.



(a) Standard Neural Net



(b) After applying dropout.

Advantages:

- + Easier configuration and better generalization result compared to L-norm regularization
- + Create an ensemble of models to address overfitting issue.

Recommendation:

We suggest to use dropout and regularization separately to understand their individual contribution.

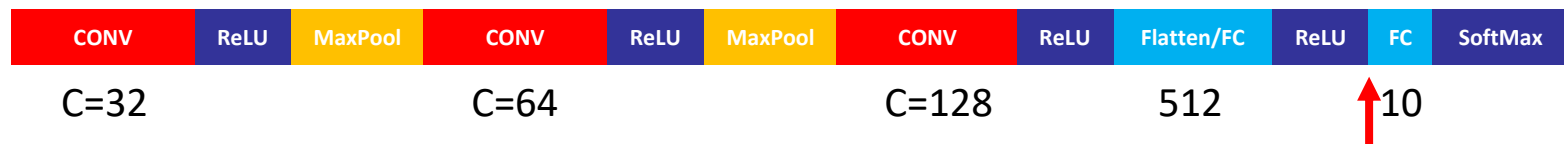
The combined effects might be complicated and need further study.

Dropout

- The **dropout probability p** is the probability to drop a neuron and its corresponding connections in the deep neural networks.
- Dropout is only enabled during the **training** phase. During the testing phase, dropout is disabled.
- Under most cases, we should dropout **no more than half** of the neurons in a DNN layer.

Dropout: position

- Traditionally, dropout is only used in fully connected layers with $p=0.5$. Recent work suggests that we can use dropout in the convolutional layers with small p values.
- Dropout is usually inserted between the FC layers. More specifically, dropout is inserted after the ReLU activation of the previous FC layer and before the next FC layer.
- **Do NOT add dropout in the last layer. It will result in information loss on predicted logits.**



Dropout: training vs. testing

- Neurons are randomly dropped with probability p during training.

$$\text{Output } Y = f(\text{Input } X, Z; W)$$

W : weight parameters.
 Z : random masks enabled during dropout.

- During testing, we average the randomness by computing the expectation over random masks to get stable outputs.

$$Y = f(X, Z; W) = E_Z[f(X, Z; W)] = \int_Z p(Z) f(X, Z; W)$$

Since each neuron has the same dropout probability p , we approximate the above integral by:

$$Y = f(X, Z; W) = (1 - p)f(X, Z; W)$$

As such, we need scale the activations by $(1-p)$ during testing.

Dropout: training vs. testing

Workflow: Dropout

Inputs: current layer activation α ; dropout probability p

Training:

- Step 1: Randomly select activations from α with probability p for dropout.
- Step 2: Set the dropped activations to 0.
- Step 3: Pass the activations (many of them have been dropped to 0) to the next DNN layer.
- Step 4: Perform back propagation (many of activations are dropped to 0).

Testing:

- Step 1: Scale the activations α by $(1-p)$.
- Step 2: Pass the scaled activations α to the next DNN layer.

However, scaling during testing introduces extra computational cost, which leads to a lower inference speed.

Inverted dropout: training vs. testing

Workflow: inverted dropout

Inputs: current layer activation α ; dropout probability p

Training:

Step 1: Randomly select activations from α with probability p for dropout.

Step 2: Set the dropped activations to 0.

Step 3: Scale the activations by $1 / (1-p)$.

Step 4: Pass the dropped activations to the next DNN layer.

Step 5: Perform back propagation (many of activations are dropped to 0).

Testing:

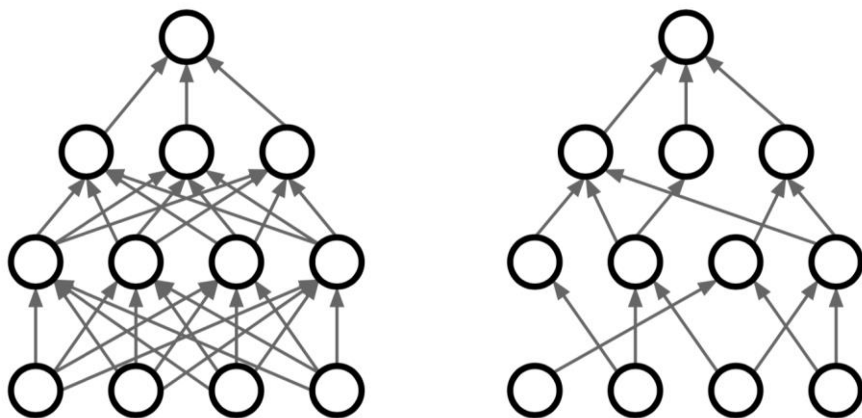
Step 1 Pass the activation α to the next layer.

Inverted dropout reduces the extra computation by moving the scaling part to the training phase.

Drop connect

- Weight connections between neurons are dropped during drop connect. Intuitively, neurons without any connections are dropped.
- Theoretical analysis suggests that drop connect has a little performance gain compared to dropout. Moreover, drop connect is not currently supported by mainstream deep learning frameworks.

Drop connect visualization



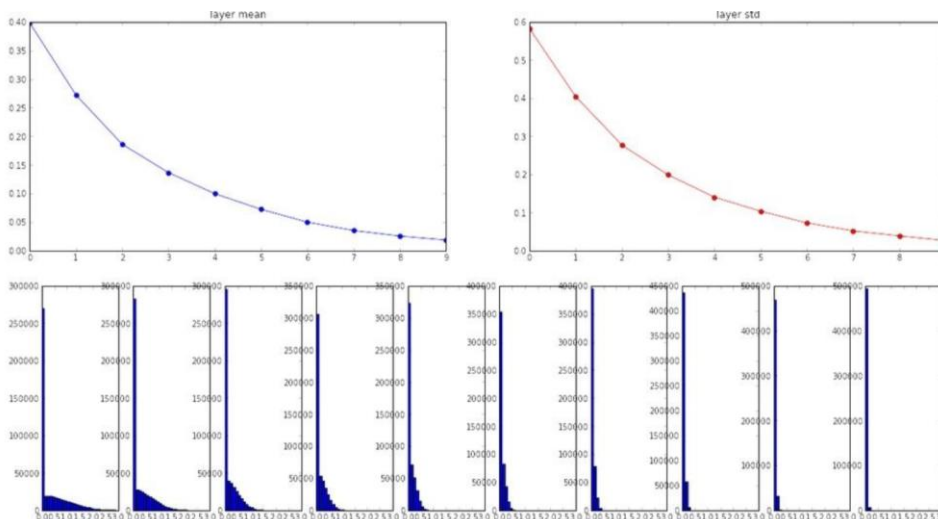
neuron	model	error(%) 5 network	voting error(%)
<i>relu</i>	No-Drop	1.62 ± 0.037	1.40
	Dropout	1.28 ± 0.040	1.20
	DropConnect	1.20 ± 0.034	1.12
<i>sigmoid</i>	No-Drop	1.78 ± 0.037	1.74
	Dropout	1.38 ± 0.039	1.36
	DropConnect	1.55 ± 0.046	1.48
<i>tanh</i>	No-Drop	1.65 ± 0.026	1.49
	Dropout	1.58 ± 0.053	1.55
	DropConnect	1.36 ± 0.054	1.35

Table 2. MNIST classification error rate for models with two fully connected layers of 800 neurons each. No data augmentation is used in this experiment.

Batch normalization

- Training DNNs is complicated because the distribution of each layer's inputs changes, as the activations of the previous layers change during the training process.
- Maintaining the similarity between the distribution of activations across layers is desired.
- Ideally, we prefer a **Gaussian distribution with zero mean and unit variance**.

Activation distribution of a 10-layer neural network



Neural networks learn the best when the inputs have zero mean and unit variance.

However, in this 10-layer network, activations are unscaled and has completely different distribution. This makes the DNN very difficult to train.

Batch normalization: training

Goal: Make the activations have zero mean and unit variance in each layer.

In training phase, we accumulate **moving averages and variances** for each neurons as well as learning trainable **scale and bias parameter** in the batch normalization layer.

Advantages:

- + DNNs can be trained at a **larger learning rate**
- + Acts as a form of **regularization**
- + DNNs are less sensitive to **weight initialization**.

Problems:

- Extra computation.
- Dropout breaks the distribution created by batch normalization.

Input: values of x over a mini-batch.

Output: $\{y_i = BN_{\gamma, \beta}(x_i)\}$

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

μ_B : mean of the current mini-batch

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

σ_B^2 : variance of the current mini-batch

γ : trainable scale parameter

β : trainable bias parameter.

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

ϵ : Small value to prevent zero

division. A good starting point is 1e-5 for small datasets and 1e-3 for large

$$y_i = \gamma \hat{x}_i + \beta$$

datasets.

Batch normalization: layer position

- Batch normalization layer (BN) is inserted before the non-linearity activations.



CONV-RELU-CONV-RELU



CONV-BN-RELU-CONV-BN-RELU

- Recall that dropout layers are added after non-linear activation.



CONV-BN-RELU-DROPOUT-CONV-BN-RELU-DROPOUT

Empirically, batch normalization gives about 2% performance gain under the same training condition.

Hyperparameter optimization

- Hyperparameters are non-trainable parameters which is configured manually to control the behavior and performance of DNNs.

After DNN architecture is fixed, important hyperparameters include:

- **Learning rate**
- Batch size
- Regularization/Dropout rate
- Optimizer

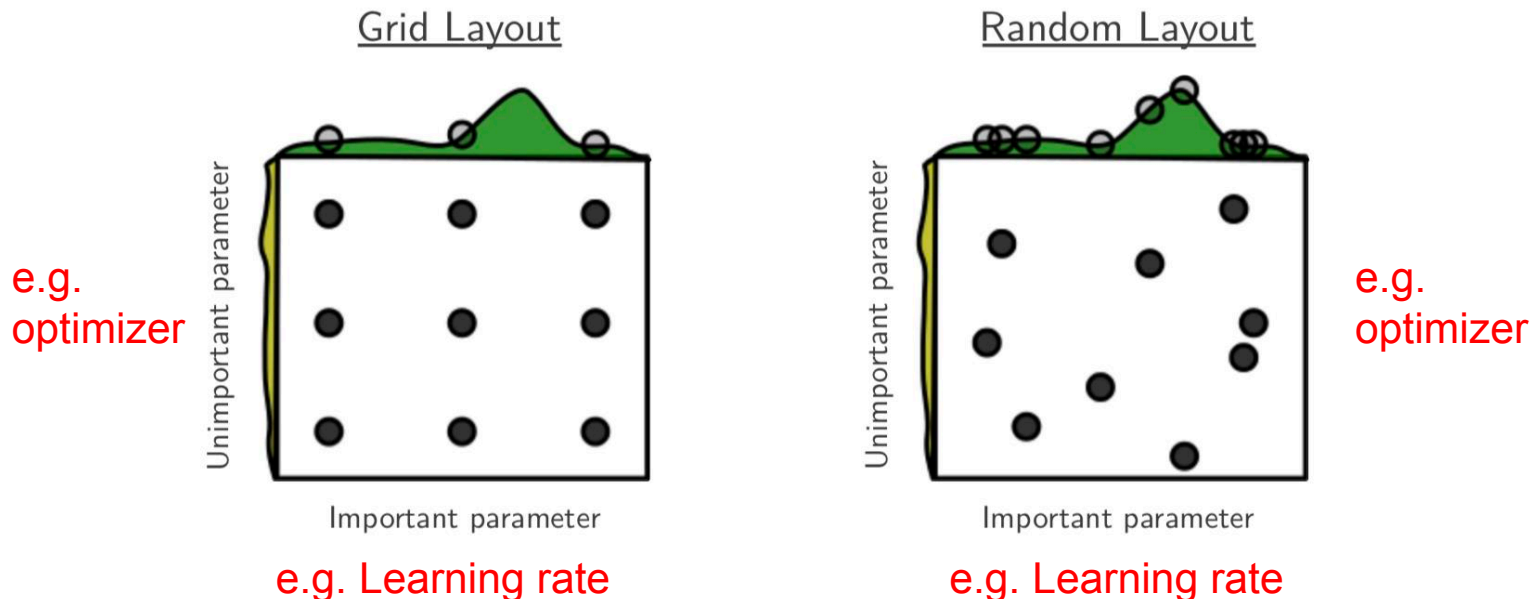
Learning rate is the most sensitive hyperparameter.

If only one hyperparameter can be tuned, tune the **learning rate**.

Hyperparameter optimization

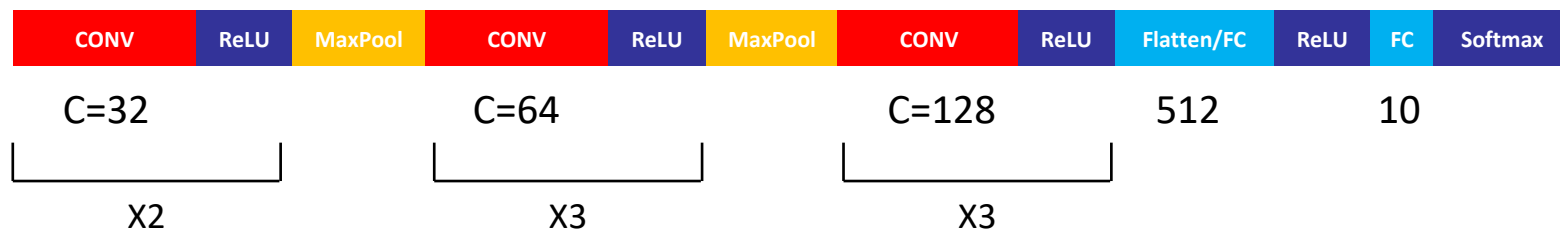
Grid Layout vs. Random Layout

The grid layout evaluates each hyperparameter with equal importance. However, random layout **evaluates important hyperparameter more frequently** thus yields better results with limited tuning trials.



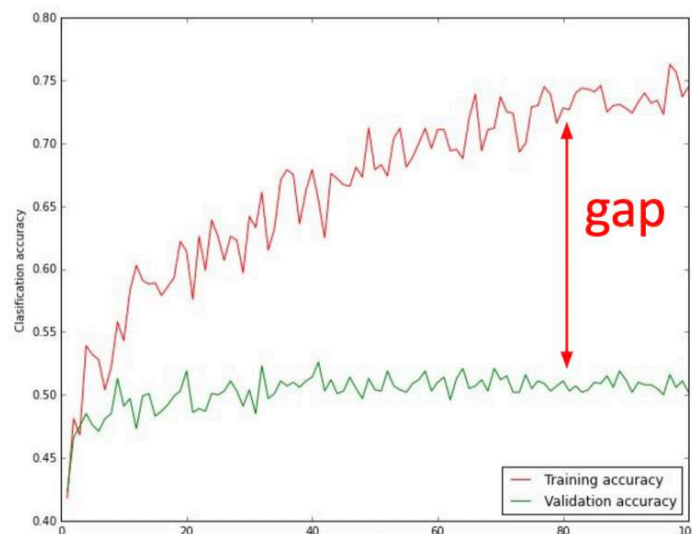
Tuning DNN architecture

DNN architecture is the most **abstract** hyperparameter.
Follow the design guidelines of convnet design:



When should add/delete layers/channels in a neural architecture?

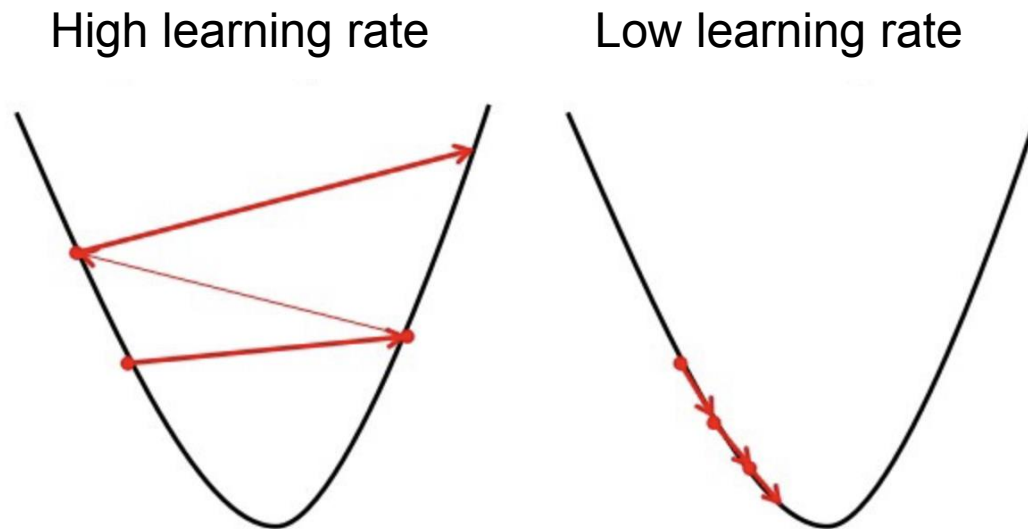
- **Decrease** model capacity by reducing layers/channels if generalization **gap** is large.
- **Increase** model capacity by increasing layers/channels if both training and validation **accuracy is low**.



Tuning learning rate

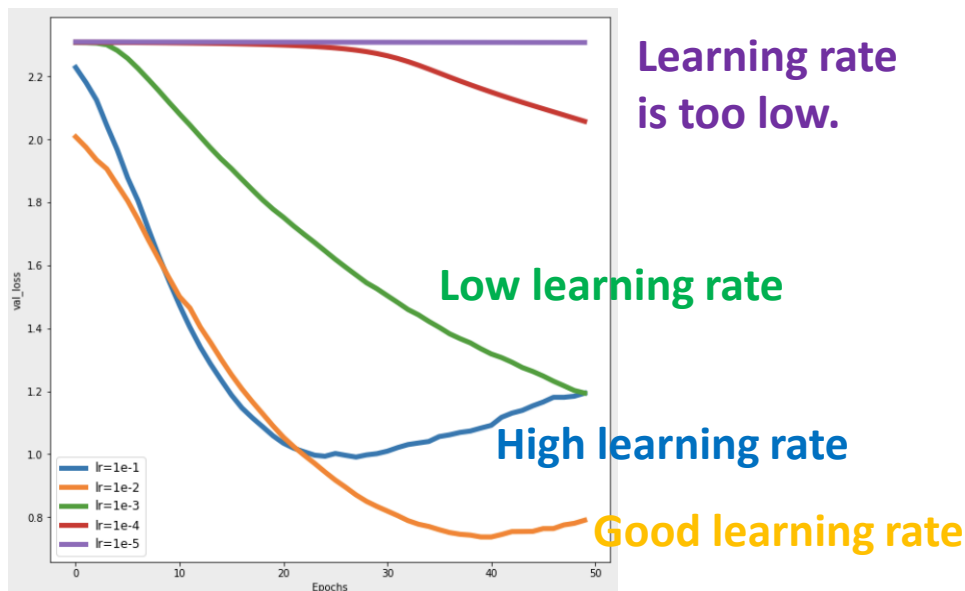
Learning rate is **the most important** hyperparameter to configure.

- **High** learning rate will lead to **divergence**.
- **Low** learning rate **slows** down the learning process and **wastes** a lot of time during the training process.



Tuning learning rate

- Under most cases, a good starting point for learning rate with **SGD with momentum** can be **0.01**.
- **Increase** the learning rate if the loss is **decreasing slowly**. Or **decrease** the learning rate if the loss **oscillates drastically or even goes up**.

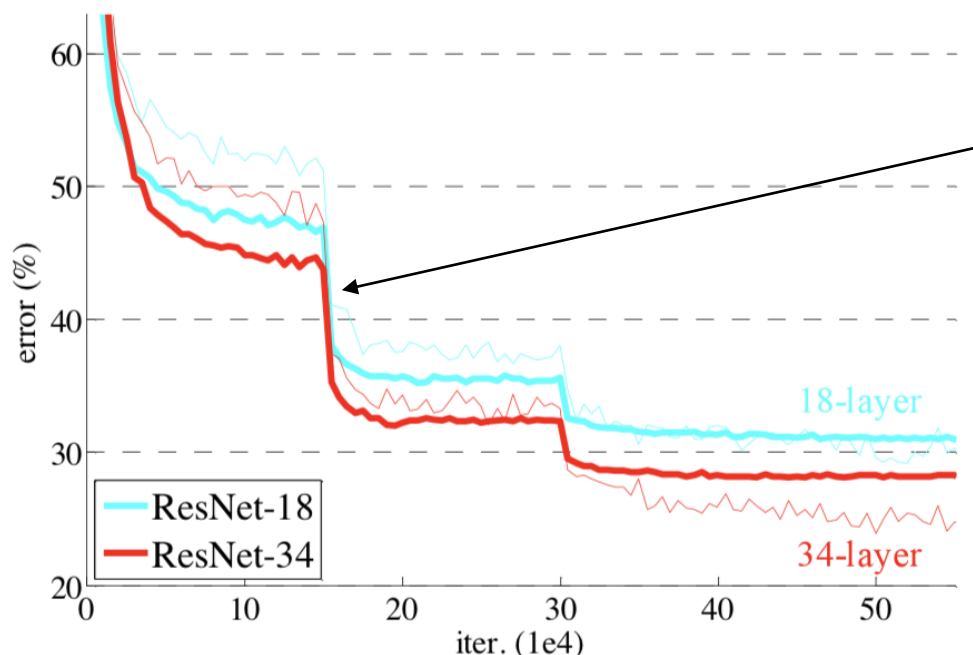


For other optimizers, the default learning rate may change due to different optimization mechanisms.

Generally, preferable learning rate reduces with the complexity of optimization algorithm. For example, a good starting point for learning rate with **Adam** optimizer is **1e-3**.

Tuning learning rate

Learning rate needs to be scheduled as the training process progresses. Usually, we **decay** the learning rate for every a few epochs during training.



Learning rate decay happens here

Recommended learning rate schedule: Decay by 0.1 when validation accuracy plateaus.

Learning curve of ResNet. Bold curve indicates validation error, thin curve indicates training error.

Tuning regularization

For L2 regularization:

- We recommend to use regularization strength $\lambda = 5e - 4$ as a good start for CNN training.
- We recommend to reduce the regularization strength if your CNN architecture is small.

For dropout:

- Dropout layer with **probability 0.5** is commonly used after each fully connected layer except the final output layer.
- You can try to add dropout layer with **probability 0.05** after each convolutional layer or pooling layer.

Tuning optimizer

- We recommend to use **Adam** optimizer as a good approach for sanity check. If Adam optimizer does not work on your network, there must be some problem with your code or neural network design.
- **SGD momentum** optimizers are usually used to reach state-of-the-art results. But be patient, they may take longer time to reach the optimal solution.

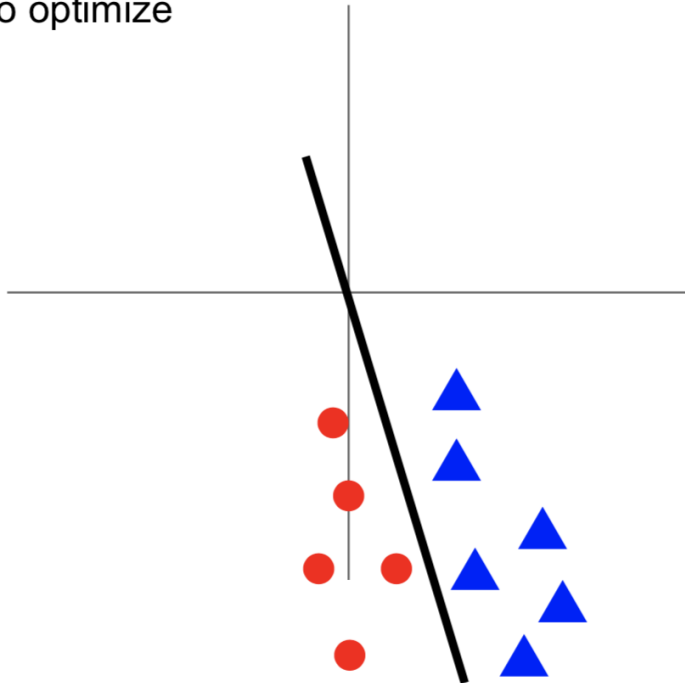
Tuning weights initialization and normalization

- Use **Xavier normal** initialization as a good start.
- Unless instructed by the original paper/repo, DO NOT use any form of uniform initialization since it will lead to inferior performance.
- With the introduction of **batch normalization**, neural networks are less sensitive to initialization.

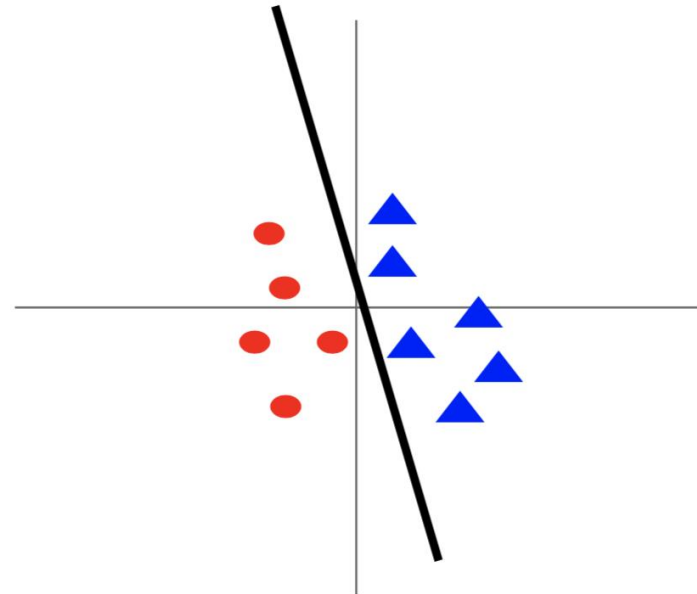
Data normalization

- Normalized activations are the best for DNN layers to learn. Usually, we normalize the input data to have zero mean and unit variance.

Before normalization: classification loss very sensitive to changes in weight matrix; hard to optimize



After normalization: less sensitive to small changes in weights; easier to optimize

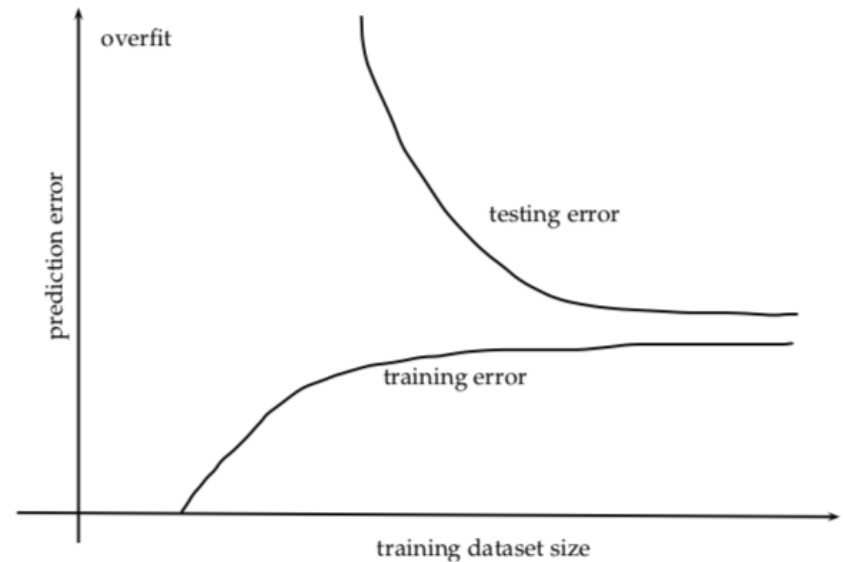


Data augmentation

- Data augmentation increases the amount of training data to reduce generalization error.
- It is also a form of 'regularization' to prevent neural networks doing too well on the training dataset.

Note: data augmentation is only enabled in the training process. It is recommended not to perform data augmentation on validation/test dataset.

Make sure the images after data augmentation makes sense.

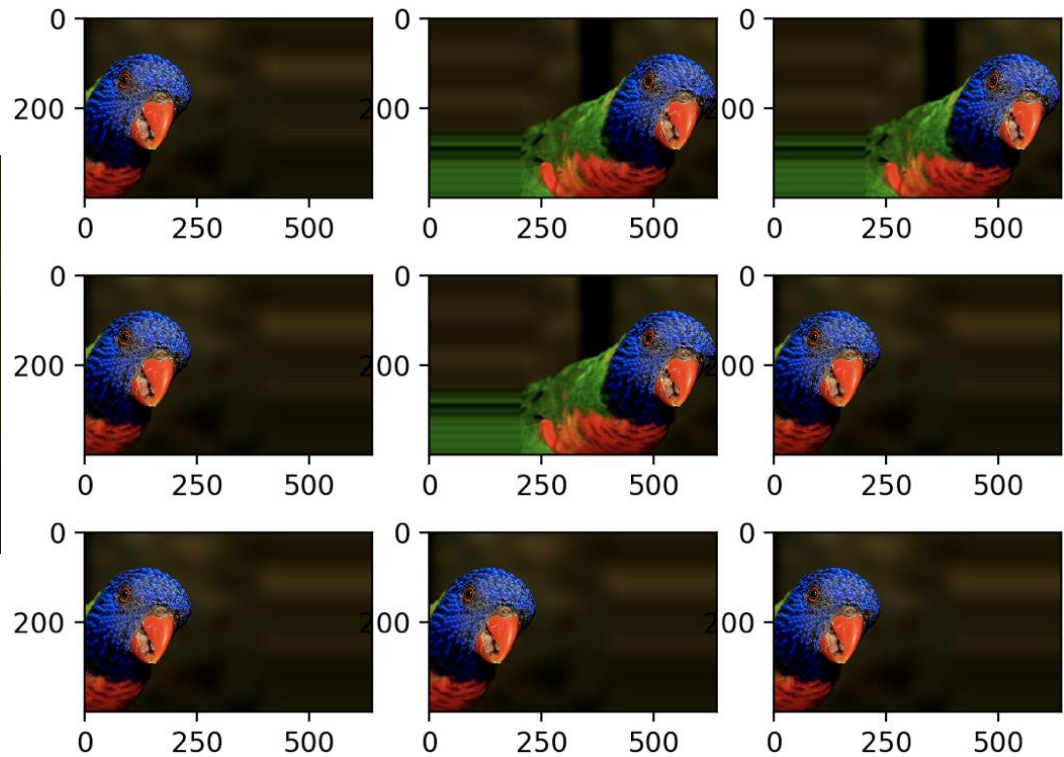


Common data augmentation for images:

- Random horizontal/vertical shift
- Random horizontal/vertical flip
- Random brightness change

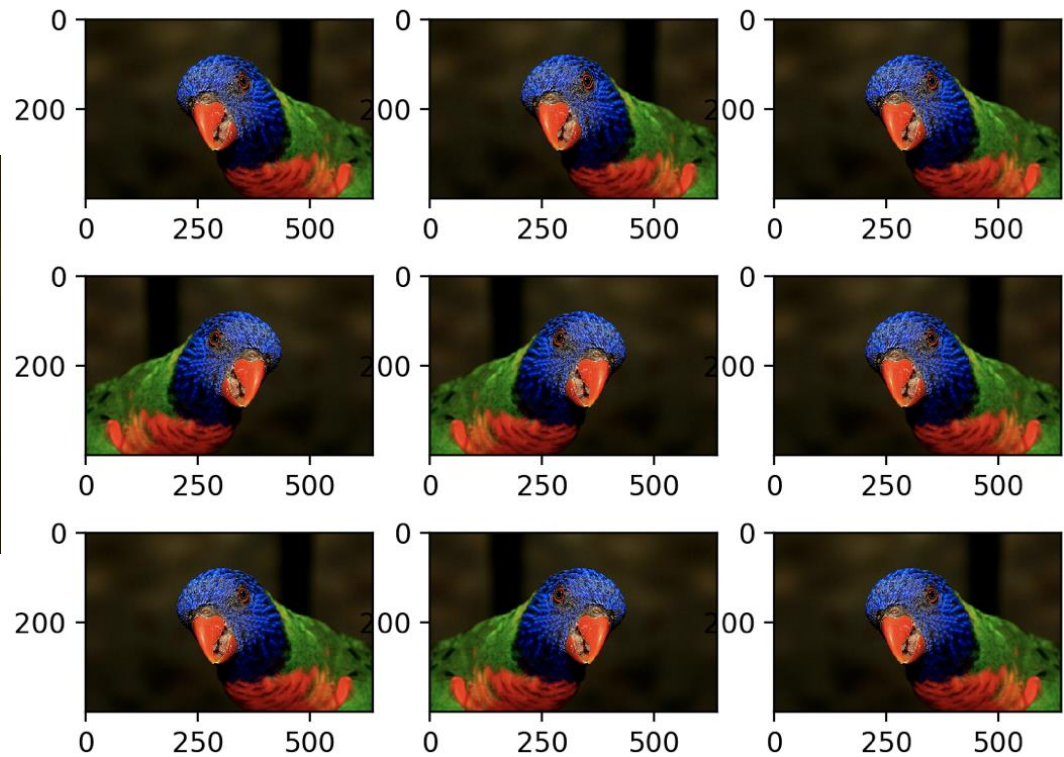
Data augmentation

- Random horizontal shift



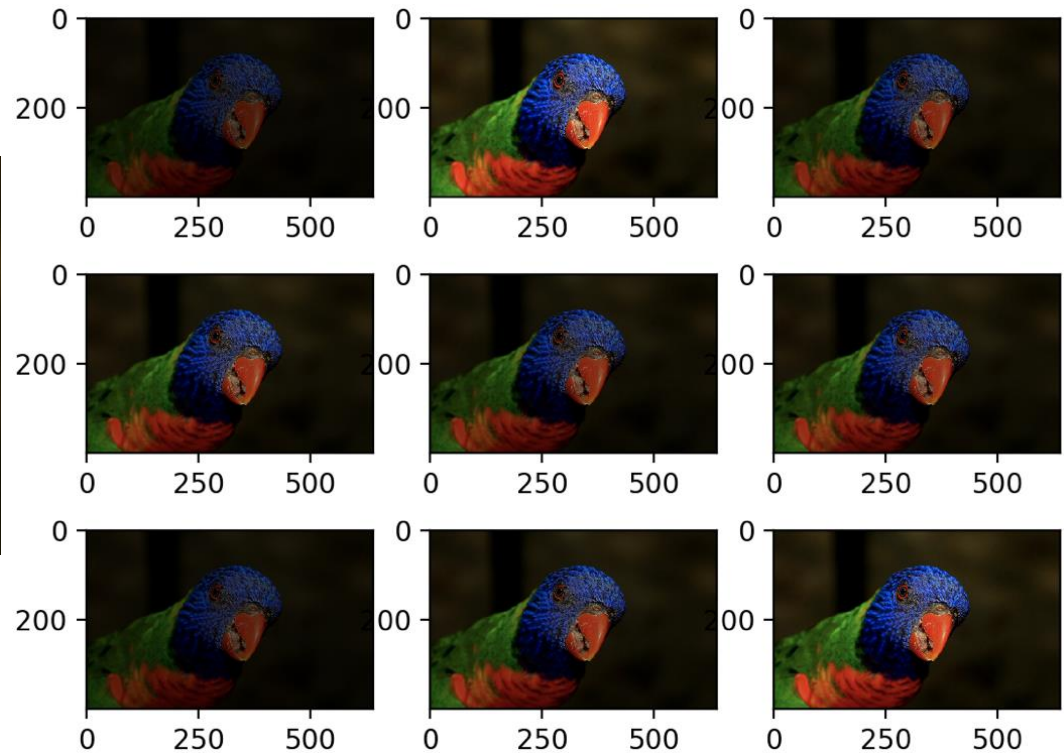
Data augmentation

- Random horizontal flip



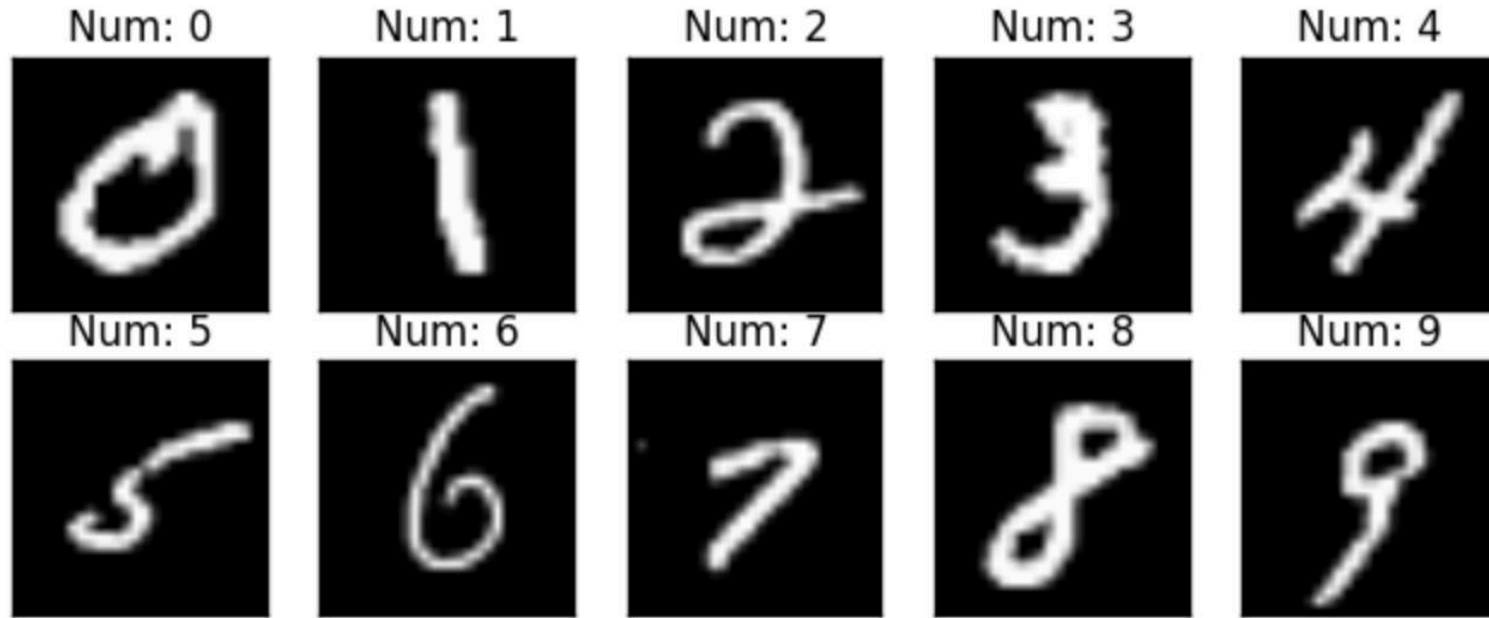
Data augmentation

- Random brightness change



Data augmentation

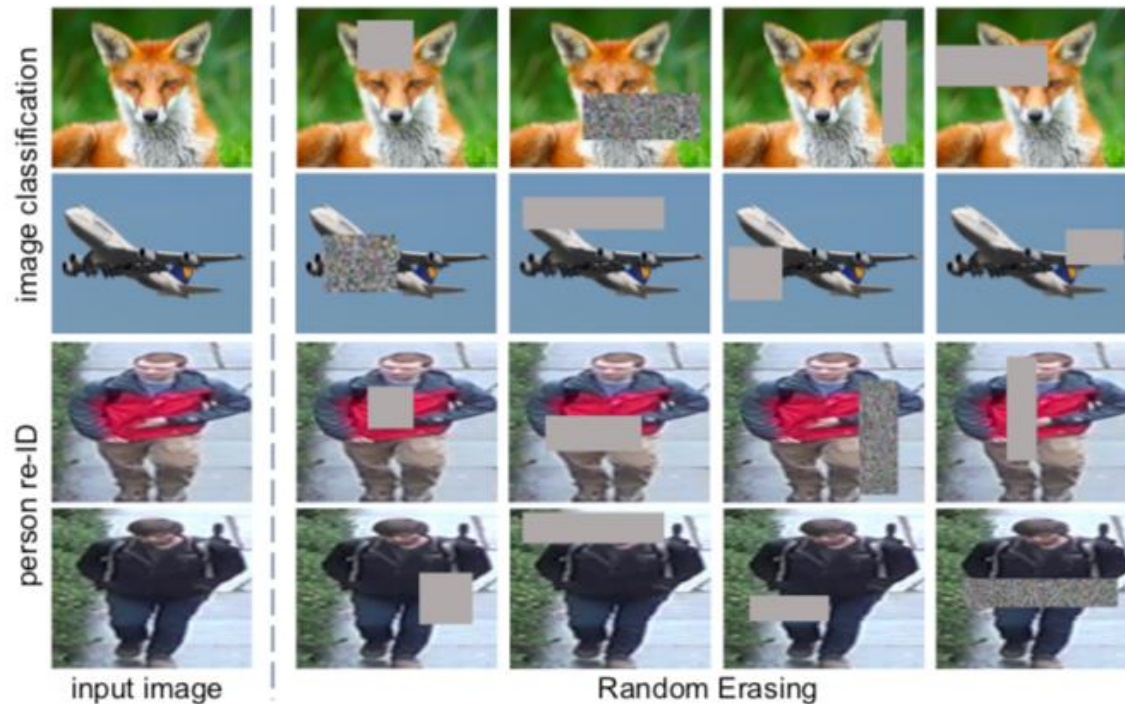
- Make sure your data augmentation method make sense!



- For example, it is unreasonable to use random vertical flip during MNIST training as digit **6** and digit **9** may look similar.

Random erasing

- Random erasing uses a random mask to occlude images to aggressively augment data during the training process. It is a complementary method to both existing data augmentation methods and regularization methods.



In this chapter, we learned:

- DNN setup for training
 - Activation function
 - Neural network design
- Optimization
 - SGD-based optimizers
 - Adagrad, RMSprop, Adam
 - How to choose the optimizer
- Weight initialization
 - Xavier uniform/normal Initializer
 - Variance scaling
 - How to choose weight initialization

In this chapter, we also learned:

- Regularization
 - L-norm
 - Dropout
 - Batch normalization
- Hyperparameter optimization
 - Learning rate
 - Model capacity
 - Tuning common hyperparameters
- Data preprocessing
 - Normalization
 - Data augmentation
 - Random erasing